

SATELLITE CHANGE INTELLIGENCE

Deep Learning-Based Satellite Image Change Detection Using
Siamese U-Net with Attention

PROJECT DOCUMENTATION

A complete technical reference: dataset, architecture, training, evaluation, advanced analysis capabilities,
and honest limitations.

Repository: [harsha282004/Satellite-Image-Change-Detection-System](#)

Dataset: LEVIR-CD (Chen & Shi, 2020)

Framework: PyTorch 2.6.0 (CUDA 12.4) · Dashboard: Streamlit 1.62.0

This document was generated from the project's actual source code, configuration files, training logs, and measured result files. Every numeric value is sourced from the repository; values that were not recorded are explicitly marked as such rather than estimated.

Table of Contents

1. Introduction	6
1.1 Problem statement	6
1.2 Motivation	6
1.3 Objective	6
1.4 Proposed solution	6
1.5 What this document is, and is not	6
2. Dataset	7
2.1 Dataset name and source	7
2.2 Dataset size and composition (verified, not assumed)	7
2.3 Dataset characteristics	7
3. Deep Learning Problem Formulation	9
3.1 Task type	9
3.2 Input and output	9
3.3 What this model does and does not do	9
4. Technology Stack	10
5. Deep Learning Concepts Used in This Project	12
5.1 Foundational concepts	12
5.2 Architecture concepts	12
5.3 Comparison and output concepts	13
5.4 Training concepts	14
6. Model Architectures	16
6.1 Architecture summary table (all real, measured)	16
6.2 Baseline U-Net	16
6.3 Siamese U-Net (diff / concat / diff+concat)	17
6.4 Siamese U-Net plus Attention (the project's recommended model)	17
6.5 Transformer-based architecture (research comparison only)	18
7. Why a Siamese U-Net Architecture	19
8. Attention Mechanism	20
8.1 What attention means in deep learning	20
8.2 Where attention is applied in this project	20
8.3 What information it emphasizes, and why this can help change detection	20
8.4 Difference between the ordinary and attention-gated U-Net in this project	20
9. Training Process and Configuration	21

9.1 The complete training pipeline	21
9.2 Training configuration (real values, per experiment)	22
10. Loss Functions	23
10.1 Binary Cross-Entropy (BCE)	23
10.2 Dice Loss	23
10.3 BCE + Dice (the project's chosen loss)	23
10.4 Alternative losses tested (and why they were not adopted)	23
11. Evaluation Metrics	24
11.1 Why accuracy alone is not sufficient for this task	24
11.2 Limitations of these metrics, as they apply here	24
12. Actual Model Performance	26
12.1 Benchmark performance (recommended model, held-out LEVIR-CD test set, 128 pairs)	26
12.2 Live / user-upload prediction (no ground truth, no accuracy figure)	26
13. Experiment Comparison	27
13.1 Baseline experiment	27
13.2 Siamese comparison-mode ablation	27
13.3 Attention comparison	27
13.4 Advanced training strategy	28
13.5 Loss-function experiments	28
13.6 Hyperparameter experiments	28
13.7 Architecture comparison: Transformer	28
13.8 Threshold optimization	29
13.9 Robustness testing	29
13.10 Which model performs best, and why	29
14. Epochs and Training Time (Consolidated)	30
15. The Prediction Mask	31
16. Threshold Optimization	32
16.1 What is a decision threshold, and why 0.5 is the common default	32
16.2 How threshold optimization was performed in this project	32
16.3 Effect on the test set, and the honest conclusion	33
17. Region-Level Analysis	34
17.1 How regions are extracted	34
17.2 Noise filtering	34
17.3 Real, measured region export result	34

18. Change Severity Analysis	36
18.1 What the severity score represents	36
18.2 What the user learns from it, and its limitations	36
19. Geospatial Analysis	37
19.1 What is geospatial analysis, in general	37
19.2 What this project implements	37
19.3 Why geospatial analysis matters, and what it tells the user beyond pixels	37
19.4 Real, measured geospatial result	37
19.5 Limitations, stated plainly	37
20. Multi-Temporal Analysis	39
20.1 What is multi-temporal analysis	39
20.2 What this project actually implements	39
20.3 Real, measured multi-temporal result	39
21. Real-World and Live Detection Workflow	40
21.1 The live upload-and-detect workflow	40
21.2 The real-world Sentinel-2 demonstration	40
21.3 Limitations that apply to any real-world or live-upload imagery	40
22. Failure Cases	41
22.1 What is a failure case, and why it matters	41
22.2 Documented failure case: false positives on a genuinely no-change scene	41
23. Input Validation	42
24. Why Streamlit	43
25. The Dashboard	44
26. Development History (Condensed)	45
27. Real-World Applications	46
27.1 Currently supported capability	46
27.2 Potential future application (not currently supported)	46
28. Limitations	47
28.1 Dataset limitations	47
28.2 Model / training limitations	47
28.3 Evaluation limitations	47
28.4 Domain shift: benchmark versus real-world (the largest gap found)	47
28.5 Multi-class change detection: not implemented, and why	48
28.6 Real-world input validation: real checks, deliberately not more than that	48

28.7 Not implemented (explicitly, never implied as working)	48
29. Future Scope	49
30. Complete System Workflow	50
31. Project Interview and Viva -- Questions and Answers	51
31.1 Problem, dataset, and task	51
31.2 Architecture	51
31.3 Output, probability, and thresholding	52
31.4 Evaluation metrics	53
31.5 Loss functions and optimization	53
31.6 Results	54
31.7 Region, geospatial, and temporal analysis	54
31.8 Errors, limitations, and engineering	55
32. Explain This Project in Two Minutes	56
33. Explain This Project to a Non-Technical Person	56
34. Technical Architecture Summary	57
35. Current Project Status	58
36. References	59

1. Introduction

1.1 Problem statement

Detecting meaningful change between two satellite images of the same geographic area, taken at different points in time, is a core remote-sensing problem with direct applications in urban monitoring, infrastructure tracking, and land management. Manually comparing before/after satellite imagery at scale is slow, inconsistent, and does not scale to the volume of imagery now available from modern Earth-observation satellites. This project addresses the specific, well-defined version of this problem for which a labeled benchmark dataset exists: given a pair of registered before/after satellite image tiles, automatically produce a pixel-level map of where **building change** occurred.

1.2 Motivation

Building change (new construction, demolition, or major structural modification) is one of the most visually and economically significant categories of land-use change, and is the category for which a rigorously constructed, peer-reviewed, publicly available benchmark dataset exists — LEVIR-CD (Chen & Shi, 2020). Building a deep-learning system for this well-defined, well-labeled problem first, rather than attempting a broader but unsupported multi-category change detector, allows every claim in this project to be backed by measured evidence rather than by an untrained or unlabeled capability.

1.3 Objective

Given two satellite images of approximately the same geographic region taken at different dates (a **before** image and an **after** image), produce a pixel-level binary change mask, extract and quantify the changed regions, and present the result through an interactive interface — first as a rigorously evaluated benchmark model on held-out test data, and separately as a clearly-caveated demonstration on independently sourced real-world satellite imagery.

1.4 Proposed solution

A **Siamese U-Net** convolutional neural network: a shared-weight encoder processes the before and after images independently, their resulting feature maps are explicitly compared at multiple scales, and a U-Net-style decoder converts the compared features into a pixel-level change probability map. An optional attention mechanism (Attention U-Net-style gates) was added and measured as a research improvement on top of this base architecture. The trained model is served through a Streamlit dashboard that also adds region-level statistics, an analytical severity score, geospatial (real-coordinate) analysis, and multi-date analysis on top of the model's raw pixel predictions.

1.5 What this document is, and is not

Documentation, not a redesign proposal

This document describes the project exactly as it currently exists in the repository. It does not propose, imply, or introduce any change to the trained models, datasets, checkpoints, or evaluation results. Every metric quoted here was measured before this document was written; nothing was measured or tuned in order to produce this documentation.

2. Dataset

2.1 Dataset name and source

LEVIR-CD (Chen & Shi, 2020), "A Spatial-Temporal Attention-Based Method and a New Dataset for Remote Sensing Image Change Detection," published in *Remote Sensing*, 12(10):1662. Official project page: justchenhao.github.io/LEVIR. This is the sole dataset used to train and evaluate every model in this project.

The official distribution is hosted on Google Drive and Baidu Drive — interactive cloud-drive folders with no stable unauthenticated direct-download URL suitable for reliable scripted acquisition. The project instead used a documented distribution mirror: the Hugging Face dataset repository **satellite-image-deep-learning/LEVIR-CD**, which exposes the same official train/val/test archives as individually downloadable, integrity-checked files. This is recorded as a mirror, not as an alternate source — the underlying dataset and its citation obligations remain Chen & Shi (2020).

2.2 Dataset size and composition (verified, not assumed)

Verified with `scripts/verify_dataset.py` against every extracted file — dimensions, channel counts, pairing, and corruption were checked file-by-file, not sampled.

Split	Before (A)	After (B)	Mask (label)	Paired	Corrupted	Dimensions	Changed-pixel fraction
Train	445	445	445	445	0	1024×1024	4.59%
Validation	64	64	64	64	0	1024×1024	4.20%
Test	128	128	128	128	0	1024×1024	5.09%
Total	637	637	637	637	0	—	—

Source: docs/DATASET.md, outputs/metrics/dataset_verification.json (scripts/verify_dataset.py). 637 total pairs exactly matches the official LEVIR-CD dataset size.

- 637 bitemporal image-pair samples, each a matched (before, after, binary-mask) triplet.
- Image size: 1024 × 1024 pixels, 0.5 m/pixel ground resolution, sourced from Google Earth.
- Temporal span: 5–14 years between the two images in a pair (imagery captured 2002–2018).
- 31,333 individually annotated building-change instances across the dataset (as documented by the dataset authors).
- 20 regions across several Texas cities (Austin, Lakeway, Bee Cave, Buda, Kyle, Manor, Pflugerville, Dripping Springs, and others).
- Data format: 3-channel RGB PNG for before/after images; single-channel ("L" mode) PNG for the ground-truth mask.

2.3 Dataset characteristics

Ground-truth masks are binary **change / no-change** maps at the pixel level — every pixel is labeled either "building change occurred here" or "no change." There is no category label distinguishing new construction from demolition, and no label for any non-building change type (road, vegetation, water, etc.). Raw mask files are not perfectly binary: most contain only pixel values {0, 255}, but some contain anti-aliased edge values such as 156 or 254. The preprocessing pipeline explicitly thresholds every mask at 127 to produce a clean {0, 1} binary target (docs/DATASET.md, "Mask binarization note").

The official, author-provided train/validation/test split is used exactly as distributed — the project does not merge and re-shuffle the splits, which would risk leaking geographically overlapping or near-duplicate tiles between

training and test data.

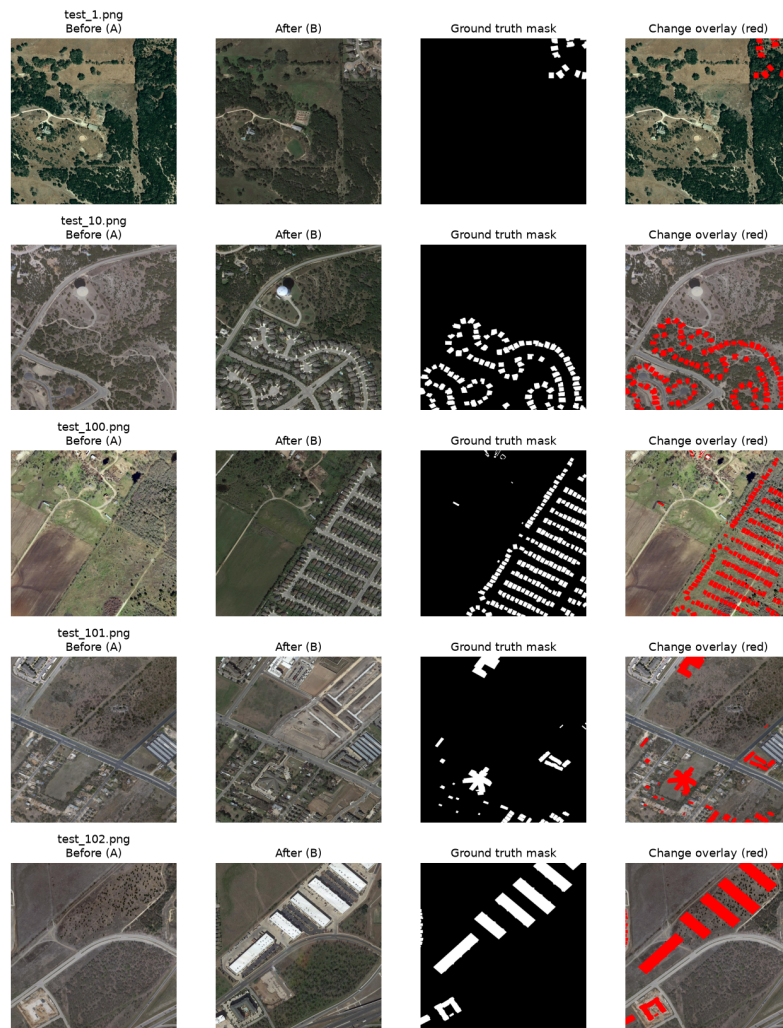


Figure 2.1 — Real samples from the LEVIR-CD test split: before image, after image, ground-truth change mask, and overlay (source: `outputs/visualizations/dataset_samples_test.png`, generated by `scripts/verify_dataset.py`).

3. Deep Learning Problem Formulation

3.1 Task type

This is binary semantic segmentation, not classification and not multi-class detection

The model performs **pixel-level binary semantic segmentation**: for every pixel in the image pair, it predicts whether that location changed (building change) or did not. It does **not** perform image-level classification (it does not output one label for the whole image), and it does **not** perform multi-class change classification — LEVIR-CD provides only a single "changed / not changed" label per pixel, so the model has no basis to distinguish change types.

3.2 Input and output

Aspect	Description
Input	Two co-registered RGB images of the same location: a "before" image and an "after" image, resized to the model's configured input resolution (256×256 for every trained model in this project).
Output	A single-channel map of raw logits, the same height/width as the resized input. Applying a sigmoid function converts this to a per-pixel change probability in [0, 1]; thresholding that probability produces the final binary change mask.
What the model predicts	For every pixel: the probability that a building changed (appeared, disappeared, or was substantially modified) between the before and after image at that location.

3.3 What this model does and does not do

Capability	Supported?
Binary building-change detection (change vs. no change, per pixel)	Yes — this is the trained, evaluated task
Semantic segmentation (pixel-level prediction)	Yes
Multi-class change classification (e.g. building vs. road vs. vegetation)	No — no such labels exist in the training data
Object detection (bounding boxes as the primary output)	No — bounding boxes are derived after segmentation, from connected components of the predicted mask, not predicted directly
Change-type or land-cover classification	No
Detecting roads, trees, water, or vegetation change specifically	No — never claimed; the training labels contain only a binary building-change signal

4. Technology Stack

Only libraries that are actually imported and exercised by the project's source code are listed as "used" below. Two packages present in requirements.txt — **scikit-learn** and **plotly** — were verified (by searching the codebase for imports) to not be used anywhere in the current implementation, and are therefore not included as active technologies.

Technology	What it is	Why it is used here
Python 3.13.0	The programming language.	The language the entire project — models, training, evaluation, dashboard — is written in.
PyTorch 2.6.0 (+cu124)	An open-source deep-learning framework.	Defines and trains every neural network in this project (U-Net, Siamese encoder, attention gates, Transformer).
torchvision 0.21.0	PyTorch's companion computer-vision library.	Supporting tensor/image utilities alongside the core PyTorch models.
CUDA 12.4 / NVIDIA GPU	GPU-accelerated computation platform.	All training and inference in this project ran on an NVIDIA RTX 4050 Laptop GPU (6 GB VRAM) via CUDA, verified in the Phase 1 environment diagnostic.
NumPy 2.5.2	Numerical array library.	Underlies almost all image-array and mask manipulation throughout the pipeline.
OpenCV (opencv-python) 5.0.0	Computer-vision library.	Image resizing, contour detection for region perimeter/geometry, phase-correlation-based registration estimation.
Pillow (PIL) 12.3.0	Image loading/saving library.	Reads and writes PNG/TIFF images throughout preprocessing, dashboard uploads, and visualization export.
Pandas 3.0.5	Tabular data library.	Builds and exports comparison tables (architecture comparison, region tables) in the dashboard and scripts.
Matplotlib 3.11.1	Plotting library.	Training-curve plots, dataset sample grids, prediction-probability colormaps, robustness/threshold charts.
PyYAML 6.0.3	YAML parsing library.	Loads every experiment's configuration file (configs/*.yaml).
Streamlit 1.62.0	Python web-application framework.	Serves the interactive dashboard — upload, inference, visualization, and analysis pages.
Rasterio 1.5.1	Geospatial raster I/O library (GDAL-based).	Reads real Sentinel-2 GeoTIFF imagery, including native CRS and affine transform, for the geospatial and multi-temporal analysis features.
pystac-client 0.9.0	STAC (SpatioTemporal Asset Catalog) API client.	Searches the Earth Search catalog for real Sentinel-2 scenes by location, date, and cloud cover.
Shapely 2.1.2	Geometric-object library.	Represents detected change regions as real geographic polygons for geospatial export.
PyProj 3.7.2	Cartographic-projection library.	Reprojects detected-region coordinates between a scene's native UTM CRS and WGS84 for GeoJSON export.
GeoPandas 1.1.4	Geospatial data-analysis library (pandas + geometry).	Exports detected regions as a GeoPackage (.gpkg) file alongside GeoJSON.

Technology	What it is	Why it is used here
Folium 0.20.0	Interactive-map library (Leaflet.js wrapper).	Renders the interactive map of detected geographic change regions shown in the dashboard's Geospatial Intelligence page.
scikit-image / cv2 contour tools	Region/connected-component utilities.	`scipy.ndimage.label` and `cv2.findContours` extract and measure connected regions from the binary change mask.
Git / GitHub	Version control and remote hosting.	Every phase of development is committed; the repository is hosted at harsha282004/Satellite-Image-Change-Detection-System .
YAML / JSON	Configuration and data-interchange formats.	YAML configures every training run; JSON stores every measured metrics file, checkpoint metadata, and geospatial export.

Not actually used in the implementation

scikit-learn and **plotly** are listed in requirements.txt but are not imported anywhere in the current source code (verified by a codebase search). They are not described as active technologies above, per the instruction not to claim a dependency is used simply because it appears in a requirements file.

5. Deep Learning Concepts Used in This Project

This section explains, in beginner-friendly but technically accurate terms, every deep learning and computer-vision concept this project actually uses -- what it is, why it is needed, and how it appears concretely in this codebase.

5.1 Foundational concepts

Deep learning

A branch of machine learning that uses multi-layer neural networks to learn patterns directly from data, rather than from hand-designed rules. This project uses deep learning because manually writing rules to distinguish a new building appearing from ordinary lighting/seasonal differences across arbitrary satellite imagery is impractical -- the network learns this distinction from thousands of labeled examples.

Computer vision

The field of teaching computers to interpret visual information (images/video). This project applies computer vision to satellite imagery specifically: reading pixel data and reasoning about the physical scene it represents.

Image segmentation

Classifying every individual pixel of an image into a category, producing a full pixel-level map rather than one label for the whole image. This project segments each pixel into changed or unchanged.

Semantic segmentation

Segmentation where each pixel is assigned a meaningful class label (as opposed to grouping pixels without naming the groups). Here, the two classes are binary: change and no-change -- there is no further semantic category, since the training labels do not provide one.

Binary segmentation

Semantic segmentation with exactly two classes. This project is a binary segmentation task: every pixel is either changed (1) or unchanged (0).

Change detection

The task of identifying differences between two observations of the same scene taken at different times. Here, the two observations are a before and an after satellite image of the same location, and the detected difference is building change.

Multi-temporal image analysis

Analysis that uses more than one time-stamped observation of a scene. This project's core model uses exactly two time steps (before/after); the Temporal Analysis feature (Section 20) extends this to a sequence of more than two observation dates, analyzed as independent adjacent pairs.

5.2 Architecture concepts

Convolution

A mathematical operation that slides a small learnable filter across an image to detect local visual patterns (edges, textures, shapes). Every model in this project is built from convolutional layers -- this is what lets the network recognize building outlines and boundaries in the satellite imagery.

Feature extraction

The process by which a neural network transforms raw pixel values into increasingly abstract, useful representations (features) through successive layers. The encoder half of this project's networks performs feature extraction on the before and after images.

Encoder

The part of a segmentation network that progressively downsamples the input image while extracting features at increasing levels of abstraction (edges to textures to shapes to object-level patterns). In this project's Siamese U-Net, the encoder is applied identically to both the before and after image.

Decoder

The part of a segmentation network that progressively upsamples the extracted features back to the original image resolution, producing a full-resolution prediction map. This project's decoder reconstructs a full-resolution change mask from the encoder's compressed features.

Skip connections

Direct connections that pass feature maps from an encoder stage straight to the corresponding decoder stage, bypassing the network's compressed bottleneck. They let the decoder recover fine spatial detail (like a building's precise edges) that would otherwise be lost during downsampling. This project's U-Net decoder uses skip connections at every scale.

U-Net

A widely used encoder-decoder segmentation architecture (named for its U-shaped diagram) with skip connections linking each encoder stage to its matching decoder stage. This project's baseline model is a standard U-Net; the primary Siamese U-Net reuses the same decoder design on top of a two-branch, shared-weight encoder.

Siamese network

A network architecture in which two (or more) inputs are passed through an identical, weight-sharing sub-network, and the resulting outputs are then compared. This project's primary architecture is a Siamese U-Net: the same encoder module is called once on the before image and once on the after image, guaranteeing both are processed by exactly the same learned filters before their features are compared.

5.3 Comparison and output concepts

Difference feature representation

Comparing the before and after feature maps by taking their absolute difference, $\text{abs}(\text{feature_before} - \text{feature_after})$. This directly highlights where the two feature maps disagree, but discards the original feature values themselves. In this project's ablation study, this mode alone performed worse than the simple baseline (Section 13), a genuinely informative negative result.

Concatenation feature representation

Comparing the before and after feature maps by stacking them together channel-wise, so the decoder sees both full feature maps side by side and can learn its own way of comparing them. This project also implements a combined difference-plus-concatenation mode, which concatenates both raw feature maps and their absolute difference -- the mode used by the project's best-performing model.

Attention mechanism

A mechanism that lets a network learn to weight different spatial locations of a feature map by importance, rather than treating every location equally. See Section 8 for a full, dedicated explanation of how this project uses it.

Prediction mask

The final binary output of the model: a same-size grid of 0s and 1s marking which pixels are predicted as changed. See Section 15 for a full explanation.

Probability map

The per-pixel probability (0 to 1) that a pixel changed, before it is thresholded into a binary mask. See Section 15 and the sigmoid activation entry below.

Sigmoid activation

A mathematical function that squashes any real number into the range zero to one. This project's models output raw, unbounded numbers (logits); applying sigmoid converts each pixel's logit into an interpretable probability.

Thresholding

Converting a continuous probability map into a binary mask by choosing a cutoff value (commonly 0.5): probabilities above the threshold become changed (1), probabilities at or below become unchanged (0). This project's threshold is user-adjustable and was also optimized on the validation set (Section 16).

Binary classification / segmentation

A prediction task with exactly two possible outcomes per unit of prediction. Framed per-pixel, this project's task is binary segmentation; framed per-pixel-decision, it is a binary classification made independently at every pixel.

5.4 Training concepts

Training

The process of adjusting a network's internal parameters (weights) using labeled examples so its predictions get closer to the correct answer over time. In this project, training uses the 445 labeled LEVIR-CD training pairs.

Validation

Evaluating the model, during training, on a held-out set of examples it is never trained on -- used to decide when the model is improving and which checkpoint to keep. This project uses the official 64-pair LEVIR-CD validation split for this purpose.

Testing

A final evaluation on a separate held-out set that is used only once, after all training and tuning decisions are finalized, to report the model's real performance. This project uses the official 128-pair LEVIR-CD test split, and it is never used to choose hyperparameters, thresholds, or checkpoints.

Backpropagation

The algorithm that computes how much each network weight contributed to the prediction error, so the optimizer knows which direction to adjust each weight in. This is the standard PyTorch autograd mechanism underlying every training run in this project.

Loss function

A single number that measures how wrong the model's prediction was for a given batch of examples; training works by adjusting weights to make this number smaller. See Section 10 for the specific loss functions used and compared in this project.

Optimizer

The algorithm that uses the gradients from backpropagation to actually update the network's weights. This project uses Adam by default, with AdamW available and tested as an alternative (Section 9).

Learning rate

A number controlling how large each weight-update step is. Too high can make training unstable; too low can make it very slow to improve. This project's default learning rate is 0.0001 (1e-4), with 5e-5 and 2e-4 also tested experimentally (Section 13).

Learning-rate scheduler

A rule that automatically changes the learning rate during training, typically reducing it once progress stalls, to allow finer adjustments later in training. This project's best model uses a ReduceLROnPlateau scheduler, halving the learning rate after 4 epochs without validation-IoU improvement (Section 9).

Early stopping

Automatically ending training once validation performance stops improving for a set number of epochs (its patience), to avoid wasting time and compute. This project's best model uses early stopping with patience 10 on validation IoU; the best checkpoint seen so far is always kept regardless of when training stops.

Epoch

One complete pass through the entire training dataset. This project's models were trained for anywhere from 30 to a maximum of 100 configured epochs, depending on the experiment (Section 9).

Batch size

The number of training examples processed together in one forward/backward pass before the weights are updated. This project's default batch size is 8 (a batch size of 4 was also tested experimentally, Section 13).

Checkpoint

A saved snapshot of a model's weights (plus, in this project, the optimizer state, training epoch, validation metrics, and the exact configuration used) at a specific point in training. Every experiment saves both a best checkpoint (best validation IoU seen) and a last checkpoint (most recent epoch).

Overfitting

When a model performs well on its training data but poorly on unseen data, because it has memorized training-specific patterns rather than learned generalizable ones. This project explicitly checked for overfitting in its longer-training experiments (Section 13) and did not observe it: training and validation metrics tracked closely at every best checkpoint.

Generalization

A model's ability to perform well on data it was not trained on. This project's benchmark test-set metrics (Section 12) measure generalization within the LEVIR-CD domain; the real-world demonstration (Section 21) probes -- without a quantitative answer, since no ground truth exists there -- generalization to a different sensor and resolution.

6. Model Architectures

Every architecture below actually exists in the codebase, was actually trained, and was actually evaluated on the real held-out LEVIR-CD test set. Parameter counts, epochs, training time, and metrics are copied from the project's measured metrics files -- none are estimated.

6.1 Architecture summary table (all real, measured)

Architecture	Parameters	Best epoch	Test IoU	Test Dice	Inference (ms/pair)
Baseline U-Net	7,763,905	30	0.6234	0.7680	4.30
Siamese U-Net (diff)	7,763,041	30	0.5569	0.7154	5.52
Siamese U-Net (concat)	10,709,345	29	0.6351	0.7768	7.09
Siamese U-Net (diff+concat)	14,704,225	29	0.6442	0.7836	8.38
Siamese U-Net + Attention (30-epoch budget)	15,428,125	26	0.6560	0.7922	10.40
Siamese U-Net + Attention (100-epoch budget, recommended model)	15,428,125	68	0.7123	0.8320	approx. 10.4 (same architecture)
Transformer (research comparison)	4,054,481	27	0.3575	0.5267	3.42

*Source: outputs/metrics/architecture_comparison.json (parameters and inference time, measured together on the same hardware) and each model's own outputs/metrics/*_test_metrics.json. Inference time measured as batch=1, 5 warm-up + 50 timed forward passes, CUDA-synchronized, on the project's NVIDIA RTX 4050 Laptop GPU.*

6.2 Baseline U-Net

A standard, non-Siamese U-Net (models/unet.py, class BaselineChangeUNet). The before and after images are concatenated channel-wise into a single 6-channel input before any convolution, and this fused input passes through a conventional 4-stage encoder-decoder U-Net. It has no explicit mechanism to compare before/after features -- it can only learn an implicit comparison from the fused input. This is a deliberately simple reference point, built and evaluated before any more advanced architecture.

Field	Value
Input	One 6-channel image (before and after concatenated on the channel axis)
Output	Single-channel raw logits, same height/width as the input
Major components	4 downsampling stages (DoubleConv + MaxPool2d), symmetric 4-stage decoder
Parameters	7,763,905 (base_channels=32)
Configuration	configs/baseline.yaml -- Adam, lr=1e-4, BCE+Dice loss, batch size 8, image size 256, 30 epochs, seed 42
Best epoch	30 (fixed 30-epoch budget, no early stopping configured)
Test IoU / Dice / Precision / Recall / F1 / Accuracy	0.6234 / 0.7680 / 0.7333 / 0.8062 / 0.7680 / 0.9752
Inference time	4.30 ms/pair (measured, Section 6 comparison table)

6.3 Siamese U-Net (diff / concat / diff+concat)

The project's primary architecture family (models/siamese_encoder.py, models/siamese_unet.py). A single shared-weight encoder module is called once on the before image and once on the after image -- the same module instance, not two separately trained copies -- producing five feature-map scales for each. The corresponding before/after feature maps are then explicitly compared at every scale, using one of three configurable modes, before feeding a standard U-Net decoder.

Comparison mode	Operation	Parameters	Test IoU	Test Dice
diff	abs(before_features - after_features)	7,763,041	0.5569	0.7154
concat	concatenate before and after features	10,709,345	0.6351	0.7768
diff + concat	concatenate both features and their difference	14,704,225	0.6442	0.7836

*All three trained under the identical 30-epoch controlled protocol (configs/siamese_diff.yaml, configs/siamese_concat.yaml, configs/siamese.yaml). Source: docs/EXPERIMENTS.md, outputs/metrics/*_test_metrics.json.*

A genuinely informative negative result

The diff-only comparison mode (IoU 0.5569) performs worse than the simple, non-Siamese baseline (IoU 0.6234). Discarding the raw feature values in favor of only their difference loses more useful information than the baseline's naive channel-wise concatenation retains. This is reported honestly rather than omitted.

6.4 Siamese U-Net plus Attention (the project's recommended model)

Adds Attention U-Net-style gates (Oktay et al., 2018) to the diff+concat Siamese architecture (models/attention.py). Before each decoder skip connection is used, it is re-weighted per pixel by a gate computed from the coarser decoder context at that stage -- the network learns to suppress irrelevant regions of the skip connection and emphasize relevant ones.

Training budget	Max epochs	Actual epochs	Best epoch	Test IoU	Test Dice	Training time
30-epoch controlled comparison (Section 13 architecture study)	30	30	26	0.6560	0.7922	Not precisely recorded (timing instrumentation added later)
60-epoch budget, early stopping enabled, LR scheduler enabled	60	60	60	0.7031	0.8257	3491.9 s (58.2 min)
100-epoch budget, early stopping enabled, LR scheduler enabled (recommended model)	100	78	68	0.7123	0.8320	3253.9 s (54.2 min)

Source: docs/TRAINING.md (Phase 13), outputs/metrics/training_experiment_comparison.csv. All three share identical architecture, data, optimizer, and seed -- only the training budget and scheduling differ.

Parameters: 15,428,125 for every row above (identical architecture; only the training recipe differs). The 100-epoch-budget checkpoint (best epoch 68, checkpoint directory outputs/checkpoints/siamese_unet_diff_concat_attention_e100/) is the model used by default throughout the dashboard and is referred to as the recommended model in this document.

Metric	Value (recommended model, held-out test set)
IoU	0.7123
Dice	0.8320

Metric	Value (recommended model, held-out test set)
Precision	0.8402
Recall	0.8239
F1	0.8320
Accuracy	0.9830

6.5 Transformer-based architecture (research comparison only)

A genuinely self-attention-based Siamese encoder (models/transformer_change.py), built to test whether a Transformer-style architecture would outperform the CNN-based Siamese U-Net on this task. It patch-embeds each image into a 16 by 16 token grid, processes both branches through a shared multi-head self-attention transformer encoder, compares the resulting token grids, and decodes back to full resolution with transposed-convolution blocks. It was trained under the identical 30-epoch controlled protocol used for the CNN architecture comparison.

Reported honestly: this architecture underperforms every CNN model tested

Test IoU 0.3575 -- below even the weakest CNN variant (diff-only, 0.5569). This is consistent with Vision Transformers generally needing substantially more training data or large-scale pretraining than this project's 445 from-scratch training pairs provide. It does have the fewest parameters (4,054,481) and the fastest measured inference (3.42 ms/pair) of every architecture in this project -- a real but non-decisive trade-off. This architecture is a research comparison only and is never used as the project's primary or recommended model.

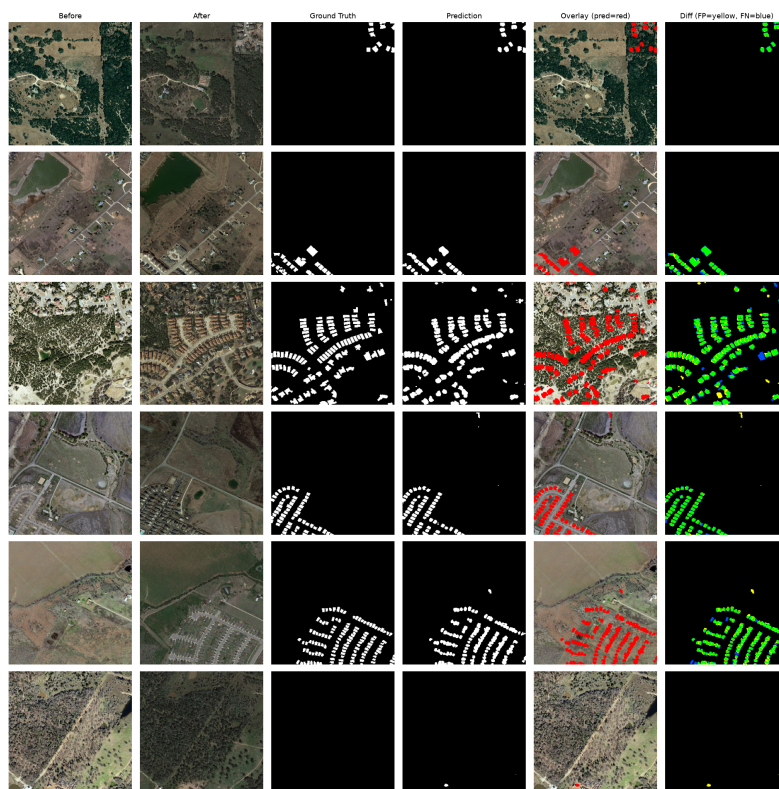


Figure 6.1 -- Real prediction grid from the recommended model on held-out LEVIR-CD test images: before / after / ground truth / prediction / overlay / difference (false positive = yellow, false negative = blue). Source: [outputs/visualizations/siamese_unet_diff_concat_attention_e100_test_predictions.png](#).

7. Why a Siamese U-Net Architecture

Change detection fundamentally requires comparing two images of the same location. A plain, single-image convolutional network has no natural way to make that comparison -- it processes one image at a time. Two images are required here specifically because change is, by definition, a relationship between a before state and an after state; neither image alone contains that information.

The naive alternative, and its problem

The simplest way to feed two images into one network is to stack them into a single 6-channel input (this project's baseline model does exactly this). The network then has to implicitly learn to separate the two images and compare them on its own, using ordinary convolutional filters that were not designed with that comparison in mind. This project's own measured results show the cost of this: the baseline U-Net has lower precision, IoU, Dice, and F1 than the Siamese architecture (Section 12) -- more false-positive noise, consistent with a harder implicit-comparison problem.

The Siamese solution: a shared encoder

A Siamese network instead passes the before image and the after image through the exact same encoder module, one at a time, with the module's weights fully shared between the two passes. Because it is a single trained module applied twice (not two independently trained encoders), the network is guaranteed to extract features from both images using identical learned filters -- so any difference between the two resulting feature maps reflects a real difference between the images, not a difference in how the images were processed. This project's tests (`tests/test_siamese_unet.py`) verify this weight-sharing directly: the encoder's parameters appear exactly once in the model's full parameter list, and calling the encoder twice on the same input produces identical output both times.

Explicit feature comparison, then decoding

Once both images have been encoded into feature maps at multiple scales, the corresponding before/after feature maps are explicitly compared (by difference, concatenation, or both -- Section 6) at every scale, and the compared features are passed through a standard U-Net decoder with skip connections to produce a full-resolution change mask. This explicit comparison step is exactly what the baseline's naive channel-stacking approach lacks.

Why this is well-suited to satellite change detection specifically

- Satellite before/after pairs are typically pre-registered (aligned to the same geographic footprint), which is the condition under which a per-pixel/per-location feature comparison is meaningful -- exactly LEVIR-CD's setup.
- Sharing the encoder halves the number of independently learned convolutional filters compared to two separate encoders, reducing the risk of the two branches learning inconsistent representations from a moderately sized training set (445 pairs).
- The explicit comparison step gives the decoder a direct, interpretable signal (difference and/or concatenation) to build the segmentation from, rather than requiring it to rediscover that comparison implicitly.

Measured, not assumed

This project did not simply assume the Siamese architecture would be better -- it measured it against the non-Siamese baseline under an identical training recipe (Section 12) and found the Siamese diff+concat model ahead on IoU, Dice, Precision, F1, and Accuracy.

8. Attention Mechanism

8.1 What attention means in deep learning

An attention mechanism lets a network learn to weight different parts of its input or internal feature maps by how relevant they are to the current prediction, instead of treating every location with equal importance. In image models, this typically takes the form of a learned per-pixel or per-region weight (a gate) that is multiplied onto a feature map, amplifying useful regions and suppressing distracting ones.

8.2 Where attention is applied in this project

This project implements additive attention gates (models/attention.py, class AttentionGate/AttentionUp), following Oktay et al.'s Attention U-Net (2018). The gates sit at every decoder skip connection: before a skip connection's feature map is concatenated into the decoder, it is first re-weighted by a gate computed from the coarser decoder context at that stage. The gate itself is a small learned function -- a sigmoid-activated combination of the skip features and the decoder's current (coarser, more semantically processed) features -- producing a per-pixel weight in $[0, 1]$ that is multiplied onto the skip connection before it reaches the decoder.

8.3 What information it emphasizes, and why this can help change detection

Skip connections carry fine spatial detail from early in the encoder, but that detail is undifferentiated -- it includes both regions relevant to the segmentation task and regions that are not. The attention gate lets the network learn, using its already-more-processed decoder context, which parts of that fine detail are actually useful for this decoding step, and suppress the rest before it is combined with the decoder's own features. For change detection specifically, this offers a plausible mechanism for the network to focus decoder capacity on plausible building-shaped regions rather than being equally influenced by every pixel of raw skip-connection detail.

8.4 Difference between the ordinary and attention-gated U-Net in this project

Aspect	Ordinary Siamese U-Net	Siamese U-Net + Attention
Decoder skip connections	Used directly, unweighted	Re-weighted by a learned gate before use
Extra parameters	--	+724,000 parameters (15,428,125 vs. 14,704,225 for diff+concat)
Measured test IoU (30-epoch controlled comparison)	0.6442 (diff+concat)	0.6560
Measured test Dice / Precision / Recall / Accuracy (same comparison)	0.7836 / 0.7982 / 0.7695 / 0.9784	0.7922 / 0.8018 / 0.7829 / 0.9791

Source: docs/EXPERIMENTS.md (Section 13 architecture comparison table). Attention improved every single metric simultaneously in this controlled comparison, not just a precision/recall trade-off.

A real, measured improvement -- not assumed from the mechanism alone

Adding attention gates improved IoU, Dice, Precision, Recall, F1, and Accuracy all at once, for a modest 4.9% parameter increase, under the project's controlled 30-epoch architecture comparison. This is the strongest single architectural improvement measured in that comparison, and it is why the recommended model includes attention.

9. Training Process and Configuration

9.1 The complete training pipeline

Implemented in `src/training/` (`train.py`, `trainer.py`, `checkpoint.py`, `logger.py`). Every experiment in this project follows the same pipeline, driven by a YAML configuration file.

- **Dataset loading.** The official LEVIR-CD train/validation/test splits are loaded as matched before/after/mask triplets.
- **Preprocessing.** Before and after images are resized (bilinear interpolation) to the model's configured input size (256x256 for every trained model here) and normalized to $[0, 1]$. The ground-truth mask is thresholded at pixel value 127 to a clean binary $\{0, 1\}$ target (`docs/DATASET.md`).
- **Data augmentation.** A paired augmentor applies identical spatial transforms (flip, rotation, scale-crop) to the before image, after image, and mask together, plus independent brightness jitter to the images only -- preserving pixel-to-pixel correspondence between image and mask.
- **Train/validation/test split.** The official, author-provided LEVIR-CD split is used as-is (445 / 64 / 128 pairs) -- never re-shuffled, to avoid geographic leakage.
- **Batch creation.** A PyTorch DataLoader groups samples into batches (batch size 8 by default, 4 in one tested variant).
- **Forward pass.** The before and after batch tensors are passed through the model, producing raw logits.
- **Loss calculation.** The configured loss function (Section 10) compares the logits against the ground-truth mask.
- **Backpropagation.** PyTorch's autograd computes gradients of the loss with respect to every model weight.
- **Optimizer update.** The optimizer (Adam or AdamW) updates the weights using those gradients and the current learning rate.
- **Learning-rate scheduling** (where enabled): a ReduceLROnPlateau scheduler halves the learning rate after 4 epochs without validation-IoU improvement.
- **Validation.** After every training epoch, the model is evaluated on the 64-pair validation split using the identical metric computation used for final test evaluation.
- **Checkpointing.** A `last.pt` checkpoint is saved every epoch; a `best.pt` checkpoint is saved whenever validation IoU improves. Every checkpoint bundles the model weights, optimizer state, epoch number, validation metrics at that epoch, and the full configuration used to produce it.
- **Early stopping** (where enabled): if validation IoU does not improve for a configured number of consecutive epochs (patience), training stops -- the best checkpoint already saved is retained regardless.
- **Final evaluation.** After training, the best checkpoint is loaded and evaluated exactly once on the held-out test split to produce the reported test metrics.

In summary, one epoch consists of: load a batch, run it forward through the model, compute the loss against the ground truth, backpropagate, update the weights, repeat for every batch in the training set, then run one full validation pass and log/checkpoint the result.

9.2 Training configuration (real values, per experiment)

Model / experiment	Optimizer	LR	Loss	Batch	Max epochs	Actual	Best	Scheduler	Early stop
Baseline U-Net	Adam	1e-4	BCE+Dice	8	30	30	30	None	No
Siamese (diff)	Adam	1e-4	BCE+Dice	8	30	30	30	None	No
Siamese (concat)	Adam	1e-4	BCE+Dice	8	30	30	29	None	No
Siamese (diff+concat)	Adam	1e-4	BCE+Dice	8	30	30	29	None	No
Siamese+Attention (30ep)	Adam	1e-4	BCE+Dice	8	30	30	26	None	No
Siamese+Attention (60ep)	Adam	1e-4	BCE+Dice	8	60	60	60	ReduceLROnPlateau	Enabled (p=10), not triggered
Siamese+Attention (100ep, recommended)	Adam	1e-4	BCE+Dice	8	100	78	68	ReduceLROnPlateau	Yes, at epoch 78
Loss variant: Focal+Dice	Adam	1e-4	Focal+Dice	8	100	58	48	ReduceLROnPlateau	Yes
Loss variant: Weighted BCE+Dice	Adam	1e-4	Weighted BCE+Dice	8	100	63	53	ReduceLROnPlateau	Yes
Loss variant: Tversky	Adam	1e-4	Tversky	8	100	49	39	ReduceLROnPlateau	Yes
Hyperparameter: LR 5e-5	Adam	5e-5	BCE+Dice	8	100	63	53	ReduceLROnPlateau	Yes
Hyperparameter: LR 2e-4	Adam	2e-4	BCE+Dice	8	100	59	49	ReduceLROnPlateau	Yes
Hyperparameter: weight decay 0.01 (AdamW)	AdamW	1e-4	BCE+Dice	8	100	65	55	ReduceLROnPlateau	Yes
Hyperparameter: batch size 4	Adam	1e-4	BCE+Dice	4	100	67	57	ReduceLROnPlateau	Yes
Transformer (research)	Adam	1e-4	BCE+Dice	8	30	30	27	None	No

Source: docs/TRAINING.md, docs/EXPERIMENTS.md, outputs/metrics/{training,loss,hyperparameter}_experiment_comparison.csv. Hardware for every run: a single NVIDIA RTX 4050 Laptop GPU (6 GB VRAM), CUDA 12.4, PyTorch 2.6.0.

Why these specific choices were made

image_size=256 (not LEVIR-CD's native 1024) and batch_size=8 were chosen for tractability on the project's 6 GB GPU. loss=BCE+Dice was chosen because the task is heavily class-imbalanced (around 4-5% changed pixels) and Dice loss is less dominated by the majority class than BCE alone. The initial 30-epoch, no-scheduler recipe was deliberately fixed across all five original architectures so the architecture comparison in Section 13 would be apples-to-apples, not confounded by different per-model hyperparameters.

10. Loss Functions

All loss functions below are implemented in `models/losses.py` and operate on the model's raw logits (not sigmoid probabilities), for numerical stability.

10.1 Binary Cross-Entropy (BCE)

BCE measures how far the predicted probability at each pixel is from the true 0/1 label, penalizing confident wrong predictions heavily. It is a standard default for binary classification/segmentation, but on its own it treats every pixel equally, which is a problem when the positive class (changed pixels) is rare -- as it is here, at roughly 4 to 5 percent of all pixels.

10.2 Dice Loss

Dice loss is derived from the Dice coefficient (Section 11), a set-overlap measure between the predicted and true changed-pixel regions: $\text{loss} = 1 - \frac{2 \times \text{intersection}}{\text{sum of prediction and target sizes} + \text{small smoothing constant}}$. Unlike BCE, Dice loss is not dominated by the large majority (unchanged) class, since it directly measures overlap of the minority (changed) region -- making it specifically useful for this project's class-imbalanced task.

10.3 BCE + Dice (the project's chosen loss)

The default and, per this project's own controlled experiment (Section 13), best-performing loss: an equal-weighted sum of BCE and Dice loss. BCE provides a smooth, stable gradient signal from the very start of training (Dice alone can be unstable early in training, when predictions are mostly wrong); Dice keeps training focused on the overlap of the rare positive class rather than letting the large majority class dominate. Combining them was a deliberate, documented choice, not a default left unexamined -- and it was subsequently confirmed, not merely assumed, by testing three alternatives against it (Section 13).

10.4 Alternative losses tested (and why they were not adopted)

Loss	What it changes relative to BCE+Dice	Why it was tried
Focal + Dice	Down-weights easy/confident pixels so training focuses on hard (uncertain or misclassified) ones	Tests whether hard-example mining helps on this task's class imbalance
Weighted BCE + Dice	Up-weights the minority changed-pixel class in the BCE term (<code>pos_weight=5.0</code>)	Tests whether directly up-weighting the rare class improves recall without an unacceptable precision cost
Tversky	Generalizes Dice with independent false-positive and false-negative weights (<code>alpha=0.3</code> , <code>beta=0.7</code> -- penalizes false negatives more)	Tests a direct, configurable lever on the precision/recall trade-off

Measured result: all three alternatives underperformed plain BCE+Dice on test IoU (0.6646, 0.6539, and 0.6322 respectively, versus BCE+Dice's 0.7123), and each converged to early stopping in fewer epochs -- reaching a worse optimum faster, not needing more training time. Tversky and Weighted BCE+Dice did produce the expected recall-favoring shift (Tversky reached the highest recall of all four variants, 0.8764, but also the lowest precision and lowest IoU), confirming the mechanisms work as designed -- they simply were not the right fix for this specific task and model. Source: `docs/EXPERIMENTS.md`, `outputs/metrics/loss_experiment_comparison.csv`.

11. Evaluation Metrics

Every metric below is computed by accumulating confusion-matrix counts (true positives, false positives, false negatives, true negatives) across an entire evaluation split before computing ratios -- not by averaging per-batch metrics, which would let near-empty-mask batches skew the result under this task's class imbalance (src/evaluation/metrics.py, class MetricAccumulator).

11.1 Why accuracy alone is not sufficient for this task

The class-imbalance problem

Only about 4.2 to 5.1 percent of pixels in LEVIR-CD are labeled changed. A model that always predicts no change at every pixel would already score above 94 percent accuracy while being completely useless. This is exactly why this project reports IoU, Dice, Precision, Recall, and F1 alongside accuracy for every result, never accuracy alone.

Accuracy

The fraction of all pixels (both changed and unchanged) correctly classified: $(TP + TN) / (TP + TN + FP + FN)$. A high value looks reassuring but, as above, is easy to achieve on an imbalanced task without detecting any real change.

Precision

Of the pixels the model predicted as changed, what fraction actually changed: $TP / (TP + FP)$. High precision means few false alarms; it says nothing about how many real changes were missed.

Recall

Of the pixels that actually changed, what fraction the model correctly predicted: $TP / (TP + FN)$. High recall means few missed changes; it says nothing about how many false alarms were produced.

F1 score

The harmonic mean of precision and recall: $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$. A single number balancing both failure modes -- useful when neither false positives nor false negatives should be optimized for in isolation.

IoU (Intersection over Union)

The overlap between the predicted changed region and the true changed region, divided by their union: $TP / (TP + FP + FN)$. This is the primary metric used throughout this project to rank models, because it directly penalizes both over-prediction and under-prediction of the changed area, and is a standard metric in the segmentation literature.

Dice score

Closely related to IoU: $2 \times TP / (2 \times TP + FP + FN)$. Mathematically, Dice and IoU are monotonic transforms of each other (Dice is always greater than or equal to IoU for the same prediction), so they always rank models identically -- this project reports both because both are standard in the literature, not because they can disagree.

11.2 Limitations of these metrics, as they apply here

- All of these are pixel-counting metrics computed against a single held-out test set from one dataset (LEVIR-CD) -- they say nothing directly about performance on a different sensor, resolution, or geography (Section 21).

- None of these metrics is a measure of calibrated confidence (Section 15) -- a model can have a high IoU while its per-pixel probability values are not individually reliable indicators of correctness.
- A single run per model/configuration was evaluated in this project (Section 22 discusses this as a limitation) -- no confidence interval or variance estimate exists for any reported metric.

12. Actual Model Performance

Two fundamentally different kinds of result, never to be confused

Benchmark performance is measured against real, held-out ground-truth labels from the LEVIR-CD test split -- these are true, quantitative accuracy figures. **Live/user-upload prediction** (the dashboard's Change Detection page, and the real-world Sentinel-2 demonstration) has no ground truth to compare against, so no accuracy metric exists for it. The dashboard keeps these visually and textually separate, and so does this document.

12.1 Benchmark performance (recommended model, held-out LEVIR-CD test set, 128 pairs)

Metric	Value
IoU	0.7123
Dice	0.8320
Precision	0.8402
Recall	0.8239
F1	0.8320
Accuracy	0.9830

These values were measured on the project's specified held-out test set (outputs/metrics/siamese_unet_diff_concat_attention_e100_test_metrics.json). The test set was never used for training or for choosing the checkpoint, threshold, or hyperparameters.

Raw confusion-matrix counts behind these figures ($128 \times 256 \times 256 = 8,388,608$ total pixels):

True positives	False positives	False negatives	True negatives
351,984	66,939	75,248	7,894,437

12.2 Live / user-upload prediction (no ground truth, no accuracy figure)

When a user uploads their own before/after pair through the dashboard, or when the project's real-world Sentinel-2 demonstration is run (Section 21), the model produces a prediction exactly as it would for any other input. **No IoU, Dice, precision, recall, F1, or accuracy can be or is computed for these predictions**, because no independently verified ground-truth change mask exists for arbitrary uploaded imagery or for the real-world Sentinel-2 scenes used. The dashboard's Change Detection page reports region counts, changed area, and prediction probability for a live upload -- it does not, and must not, present these as benchmark accuracy.

13. Experiment Comparison

This section explains why each experiment in this project was run, what was changed, what was measured, and what conclusion was reached -- including the experiments whose result was negative. Full detail: docs/EXPERIMENTS.md, docs/TRAINING.md.

13.1 Baseline experiment

Why	What changed	What was measured	Result	Conclusion
Establish a simple, non-Siamese reference point before building any more advanced architecture.	A single U-Net fed the channel-stacked before/after image.	Test IoU/Dice/Precision/Recall/F1/Accuracy on the held-out test set.	IoU 0.6234 -- a working reference point.	A valid, if simple, starting point; not sufficient on its own for a rigorous comparison.

13.2 Siamese comparison-mode ablation

Why	What changed	What was measured	Result	Conclusion
Test whether an explicit before/after feature comparison (rather than the baseline's implicit one) improves results, and which comparison operation works best.	Three feature-comparison modes (diff, concat, diff+concat) trained under the identical protocol.	Test IoU/Dice/etc. for each mode.	diff 0.5569 (worse than baseline), concat 0.6351, diff+concat 0.6442 (best of the three).	Discarding raw feature values in favor of only their difference loses more information than it gains; combining both concatenation and difference is best.

13.3 Attention comparison

Why	What changed	What was measured	Result	Conclusion
Test whether Attention U-Net-style gates improve the best Siamese configuration (diff+concat).	Added attention gates to every decoder skip connection.	Test IoU/Dice/Precision/Recall/F1/Accuracy under the identical 30-epoch protocol.	IoU improved from 0.6442 to 0.6560 -- every metric improved simultaneously.	Attention gates were adopted into the project's best-performing architecture.

13.4 Advanced training strategy

Why	What changed	What was measured	Result	Conclusion
Validation IoU was still rising at epoch 30 in every prior experiment -- test whether the attention model was undertrained.	Longer training budgets (60 and 100 max epochs) plus early stopping and a ReduceLROnPlateau learning-rate scheduler.	Test IoU at each budget, and whether overfitting appeared.	IoU rose to 0.7031 (60 epochs) and 0.7123 (100-epoch budget, stopped at epoch 78, best epoch 68) -- a +0.0563 absolute improvement over the original 30-epoch result, with no overfitting observed.	The original attention model was genuinely undertrained; training strategy mattered more than any architecture change tested. This became the project's recommended model.

13.5 Loss-function experiments

Why	What changed	What was measured	Result	Conclusion
Test whether an alternative loss improves on BCE+Dice for the now-established best training strategy.	Focal+Dice, Weighted BCE+Dice, and Tversky, each substituted for BCE+Dice with every other setting held fixed.	Test IoU/Dice/Precision/Recall/F1 for each.	All three underperformed BCE+Dice (best alternative: Focal+Dice at IoU 0.6646, versus BCE+Dice's 0.7123).	BCE+Dice remains the project's loss function; no change adopted.

13.6 Hyperparameter experiments

Why	What changed	What was measured	Result	Conclusion
Test whether learning rate, weight decay, or batch size can be improved on the established recipe.	LR 5e-5, LR 2e-4, weight decay 0.01 (AdamW), and batch size 4, each varied independently.	Test IoU for each variant.	All four underperformed the original recipe (best alternative: weight decay 0.01 at IoU 0.7028, versus the original 0.7123).	The original hyperparameters (Adam, lr=1e-4, no weight decay, batch size 8) were already well suited to this setup; no change adopted.

13.7 Architecture comparison: Transformer

Why	What changed	What was measured	Result	Conclusion
Test whether a genuinely self-attention-based (Transformer) encoder outperforms the CNN-based Siamese U-Net.	A Transformer-based Siamese encoder, trained under the identical 30-epoch controlled protocol used for the original 5-architecture comparison.	Test IoU/Dice/Precision/Recall/F1/Accuracy, parameters, and inference time.	IoU 0.3575 -- substantially below every CNN architecture tested, including the weakest one (diff-only, 0.5569).	The Transformer underperforms on this dataset size without pretraining, consistent with Vision Transformers generally needing more data or large-scale pretraining than this project's 445 training pairs provide. Reported honestly; never adopted as the primary model.

13.8 Threshold optimization

Why	What changed	What was measured	Result	Conclusion
Test whether the default 0.5 decision threshold is optimal, or whether a different threshold improves results.	A sweep of 9 thresholds (0.30 to 0.70) on the validation set only, then a single confirmatory test-set evaluation at the validation-selected threshold.	Validation IoU/Dice/Precision/Recall/F1 at each threshold; test-set result at the selected threshold versus the default.	Validation IoU varied only from 0.7131 to 0.7196 across the entire 0.30-0.70 range; the selected threshold (0.40) produced a test IoU of 0.7122, versus 0.7123 at the untuned default 0.50 -- a tie within noise.	The model is essentially insensitive to threshold choice in this range. The dashboard defaults to 0.40 for the recommended model since it is technically validation-optimal, but this section reports plainly that the difference from 0.50 is not practically meaningful.

13.9 Robustness testing

Why	What changed	What was measured	Result	Conclusion
Test how sensitive the recommended model is to realistic image perturbations that could occur between two real satellite captures.	10 real test images run through 6 controlled perturbations (Gaussian noise, +/-30% brightness, +/-30% contrast, a 5-pixel simulated misregistration shift), applied to the after image only.	Mean IoU change relative to the unperturbed prediction, for each perturbation.	Small, robust changes for Gaussian noise (+0.0098) and +30% contrast (+0.0005); substantial degradation for -30% contrast (+0.1048), a 5px shift (+0.1178), and -30% brightness (+0.1198) -- these are reported as IoU degradation values.	The model is notably sensitive to darkening, reduced contrast, and misregistration -- a real, measured vulnerability, illustrated with a saved worst-case example, not hidden.

13.10 Which model performs best, and why

The recommended model: Siamese U-Net + Attention, diff+concat, 100-epoch training budget

Test IoU 0.7123 -- the highest of every architecture and every hyperparameter/loss variant tested in this project. It is best not because of a single trick, but because of the combination of an explicit, symmetric feature comparison (diff+concat), attention-gated skip connections, and -- the single largest contributing factor, measured directly -- a training strategy (longer budget, early stopping, learning-rate scheduling) that was not present in any of the architecture-comparison-era experiments. No experiment in this project's loss, hyperparameter, or Transformer comparisons beat it.

14. Epochs and Training Time (Consolidated)

Model	Max epochs	Actual epochs	Best epoch	Training time	Best test IoU	Best test Dice
Baseline U-Net	30	30	30	Not recorded	0.6234	0.7680
Siamese U-Net (diff)	30	30	30	Not recorded	0.5569	0.7154
Siamese U-Net (concat)	30	30	29	Not recorded	0.6351	0.7768
Siamese U-Net (diff+concat)	30	30	29	Not recorded	0.6442	0.7836
Siamese+Attention (30-epoch budget)	30	30	26	Not recorded	0.6560	0.7922
Siamese+Attention (60-epoch budget)	60	60	60	3491.9 s (58.2 min)	0.7031	0.8257
Siamese+Attention (100-epoch budget, recommended)	100	78	68	3253.9 s (54.2 min)	0.7123	0.8320
Loss variant: Focal+Dice	100	58	48	40.3 min	0.6646	0.7985
Loss variant: Weighted BCE+Dice	100	63	53	43.6 min	0.6539	0.7907
Loss variant: Tversky	100	49	39	43.6 min	0.6322	0.7747
Hyperparameter: LR 5e-5	100	63	53	62.4 min	0.6560	0.7923
Hyperparameter: LR 2e-4	100	59	49	45.2 min	0.6999	0.8235
Hyperparameter: weight decay 0.01	100	65	55	45.1 min	0.7028	0.8255
Hyperparameter: batch size 4	100	67	57	47.2 min	0.6997	0.8233
Transformer (research comparison)	30	30	27	889.5 s (14.8 min)	0.3575	0.5267

*Training time for the five original (Phase 4/5/8) models was not machine-logged -- wall-clock timing instrumentation was added to the training script only in a later stage of the project. Rather than estimate a figure retroactively, this table reports "Not recorded" for those rows, exactly per the instruction not to estimate missing values. All other training-time values are exact, machine-measured wall-clock figures. Source: docs/TRAINING.md, docs/EXPERIMENTS.md, outputs/metrics/*_experiment_comparison.csv.*

15. The Prediction Mask

A prediction mask is the model's final, pixel-level output describing where change was detected: a grid the same height and width as the input image, where every pixel holds either 0 (unchanged) or 1 (changed). It is produced in three steps.

- **Raw logits.** The network's decoder outputs one raw, unbounded number per pixel.
- **Probability map.** A sigmoid function converts each pixel's logit into a probability in $[0, 1]$ -- the model's estimate that change occurred at that location (`src/inference/predict.py::predict_probability`).
- **Binary mask via thresholding.** Every pixel whose probability exceeds the configured decision threshold (0.40 by default for the recommended model, user-adjustable) is set to 1 (changed); every other pixel is set to 0 (unchanged).

The resulting binary mask is what feeds every downstream feature in this project -- region extraction, area/percentage statistics, severity scoring, and geospatial polygon conversion all operate on this mask, not on the raw probability map directly (though the probability map is also retained and used for per-region confidence statistics).

16. Threshold Optimization

16.1 What is a decision threshold, and why 0.5 is the common default

The decision threshold is the probability cutoff above which a pixel is classified as changed. 0.5 is the conventional default because it is the natural midpoint of a sigmoid output with no other information favoring one class over the other. A different threshold can be better when the two error types (false positives and false negatives) are not equally costly, or when a model's probability outputs are systematically skewed.

16.2 How threshold optimization was performed in this project

A sweep of 9 thresholds (0.30 to 0.70, in steps of 0.05) was run against the recommended model's output on the validation set only. The threshold with the highest validation IoU (0.40) was then evaluated exactly once on the test set, to confirm the result without using the test set to make the selection (src/evaluation/threshold_analysis.py, scripts/threshold_optimization.py).

Threshold	Validation IoU	Validation Precision	Validation Recall
0.30	0.7175	0.8162	0.8557
0.35	0.7189	0.8246	0.8487
0.40 (selected)	0.7196	0.8322	0.8416
0.45	0.7194	0.8389	0.8346
0.50 (untuned default)	0.7188	0.8454	0.8275
0.55	0.7184	0.8520	0.8208
0.60	0.7173	0.8586	0.8135
0.65	0.7156	0.8655	0.8052
0.70	0.7131	0.8729	0.7957

Source: outputs/metrics/threshold_analysis.csv, docs/EVALUATION.md.

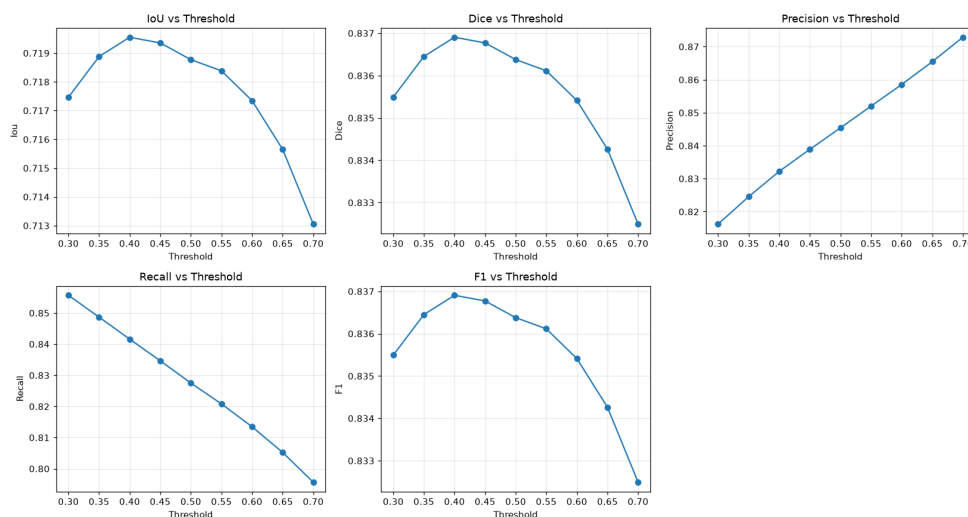


Figure 16.1 -- Real threshold-sweep curves on the validation set (source: outputs/visualizations/threshold_analysis.png).

16.3 Effect on the test set, and the honest conclusion

Threshold	Test IoU	Test Precision	Test Recall
0.40 (validation-selected)	0.7122	0.8263	0.8376
0.50 (untuned default)	0.7123	0.8402	0.8239

The threshold sweep did not actually improve test performance

The difference between the validation-selected threshold (0.7122 test IoU) and the untuned default (0.7123) is a tie within noise, not a real gain. The validation sweep shows the model's IoU is essentially flat across the entire 0.30-0.70 range (a spread of only 0.0065 IoU) -- a genuine and useful finding in itself: this model does not require threshold tuning to perform well. The dashboard still defaults to 0.40 for the recommended model since it is technically validation-optimal, and the threshold remains fully user-adjustable.

Increasing the threshold trades recall for precision (fewer false positives, more missed changes); decreasing it does the opposite (fewer missed changes, more false positives) -- the table above shows this expected monotonic pattern exactly, confirming the sweep is implemented correctly.

17. Region-Level Analysis

A raw pixel mask tells a user which pixels changed, but not how many distinct changed areas exist, how large or how shaped each one is, or how confident the model was about each one individually -- a single large connected blob and a hundred scattered single-pixel specks can produce a similar-looking mask but mean very different things. Region-level analysis converts the pixel mask into a list of distinct, individually described change regions, making the result far more interpretable to a user.

17.1 How regions are extracted

Connected-component labeling (8-connectivity, via `scipy.ndimage.label`) groups adjacent changed pixels into distinct regions. For every region, `src/analysis/regions.py` computes:

Field	Meaning
Region ID	A unique integer identifying the region, used to cross-reference the region-ID overlay image with the region table
Pixel count / area	The number of pixels in the region (and, when a real pixel size is known, the physical area)
Bounding box	The smallest rectangle (in row/column coordinates) containing the region
Width / height	The bounding box's dimensions
Perimeter	The region's boundary length, computed via <code>cv2.findContours + arcLength</code>
Aspect ratio	Width divided by height
Change density	Pixel count divided by bounding-box area -- 1.0 for a solid rectangle, lower for an irregular or sparse shape
Prediction probability (mean / max)	The model's own sigmoid output, averaged and maximized within that region -- not a separate confidence model, the same probability map described in Section 15

17.2 Noise filtering

A configurable minimum-region-size parameter (4 pixels by default) discards regions smaller than this before counting or area statistics, since at this project's effective ground sampling resolution a region under 4 pixels is below what the model can reliably distinguish from prediction noise at object boundaries. This is a user-adjustable dashboard control, not a hidden constant.

17.3 Real, measured region export result

Test image	Regions	Largest (px)	Smallest (px)	Average (px)
test_29.png	67	854	4	141.9
test_45.png	113	1450	4	164.5
test_52.png	42	322	5	64.5
test_75.png	35	491	15	148.9
test_99.png	1	41	41	41.0

258 total regions across these 5 real test images. Source: `outputs/regions/regions.csv` (`scripts/export_regions.py`).



Figure 17.1 -- Real region-ID overlay for test_29.png: each detected region's bounding box and numeric ID (source: `outputs/regions/region_ids_test_29.png`).

Terminology rule enforced everywhere

Every detected region is labeled "Detected Change Region" -- never "Building," "Road," "Vegetation," or any other semantic category. LEVIR-CD provides only a binary change/no-change label, so this project has no basis to assign a semantic category to any region (Section 3, Section 28).

18. Change Severity Analysis

Severity here is an analytical score derived from model outputs -- it is not ground truth

This project's severity score is **not** a ground-truth label, and it is **not** a validated physical damage assessment. No labeled severity data exists for this task. It is a transparent, documented, heuristic ranking formula built entirely from measurable outputs of the already-trained model and the already-extracted region geometry.

18.1 What the severity score represents

For each detected region, a 0-100 score is computed as a weighted sum of four normalized components (src/analysis/severity.py):

Component	Weight	What it measures
Area score	0.35	The region's pixel count relative to a fixed reference size (500 pixels), capped at 1.0
Probability score	0.30	The model's own mean prediction probability for that region
Density score	0.15	The region's change density (how solid/compact its shape is)
Relative-size score	0.20	The region's share of the total changed pixels in that image

The four weights sum to 1.0 and are documented, adjustable engineering defaults -- they were not derived from any labeled severity ground truth, since none exists for this task. Regions are then bucketed into four categories by score: Low (0-25), Moderate (25-50), High (50-75), and Very High (75-100).

18.2 What the user learns from it, and its limitations

The severity score lets a user triage which of many detected regions are largest, most confidently detected, most compact, and most significant relative to the rest of that image's detected change -- useful for prioritizing manual review of a long region list. It does not, and cannot, indicate anything about real-world physical damage, monetary cost, or safety impact, since no such labeled data exists anywhere in this project to validate it against.

Real, measured result on 258 regions across the 5 test images used for region export: severity scores ranged 22.3 to 69.9 (mean 44.3) -- 3 Low, 215 Moderate, 40 High, and 0 Very High. No region reached Very High in this sample, consistent with the formula's design (a region must simultaneously be large, high-probability, dense, and a large share of its image's total change to approach 100).

19. Geospatial Analysis

19.1 What is geospatial analysis, in general

Geospatial analysis means working with data that is tied to real locations on the Earth's surface -- using a coordinate reference system (CRS) so that a pixel in an image can be converted into a real latitude/longitude or a real projected (metric) coordinate, and so that shapes derived from an image (like a detected change region) can be measured in real physical units (square meters, hectares) and placed correctly on a map alongside other geographic data.

19.2 What this project implements

LEVIR-CD's training images are plain PNG files with no coordinate reference system at all -- "the model detected pixels (12, 45) to (30, 60)" is a purely image-space statement with no geographic meaning. To provide genuine geospatial analysis, this project separately fetches real, georeferenced Sentinel-2 satellite imagery (via the Earth Search STAC API, no authentication required) as GeoTIFF files that preserve their actual coordinate reference system and affine transform. A hard guard (`src/geospatial/raster.py::has_georeference`) refuses to run geospatial conversion on any image that is not genuinely georeferenced -- it will not invent coordinates for a plain upload.

- **WGS84** -- the standard latitude/longitude coordinate system used by GPS and GeoJSON.
- **UTM** -- a projected (metric), zone-based coordinate system in which distances and areas can be measured directly in meters; Sentinel-2 tiles are natively in a UTM zone.
- **Raster-to-vector conversion** -- the detected pixel regions (from the same connected-component extraction described in Section 17) are converted into real geographic polygons using the raster's actual affine transform, then their area is computed in the raster's native UTM projection (never guessed from an assumed pixel size) and reprojected to WGS84 for GeoJSON export.
- **GeoJSON / GeoPackage export** -- every detected region becomes a real geographic polygon feature, downloadable as a GeoJSON or GeoPackage file.
- **Interactive map** -- detected regions are rendered as an interactive Folium (Leaflet.js) map with per-region popups.

19.3 Why geospatial analysis matters, and what it tells the user beyond pixels

"The model detected pixels" versus "the system identifies geographically meaningful regions"

A pixel-space result ("region 3 spans rows 40-75, columns 100-130") is meaningless outside the specific resized image it came from. A geospatial result ("region 3 covers 13.65 hectares, centered at a real latitude/longitude, in UTM zone 14N") can be placed on a real map, compared against other geographic data, measured in physically meaningful units, and shared with a GIS tool -- turning a model output into something a geographic analyst can actually use.

19.4 Real, measured geospatial result

Run against a real Sentinel-2 scene over Pflugerville, Texas (583x561 pixels at 10.0 m/pixel, CRS EPSG:32614): 6 regions detected, totaling 30.89 hectares of detected-change area, computed from the raster's real UTM projection. Source: `outputs/geospatial/regions.geojson`, `docs/EVALUATION.md`.

19.5 Limitations, stated plainly

- **Resolution mismatch.** This Sentinel-2 imagery is 10 m/pixel -- 20 times coarser than the 0.5 m/pixel LEVIR-CD imagery the model was trained on. A typical house occupies a fraction of one pixel to a few pixels at this resolution.
- **No ground truth.** There is no independently labeled change mask for this real-world scene, so no accuracy metric exists for this geospatial result -- it is a real, measured output, not a validated one.
- **Registration.** No independent re-registration or alignment-quality check beyond Sentinel-2's own standard georeferencing was performed for this imagery.

20. Multi-Temporal Analysis

20.1 What is multi-temporal analysis

Multi-temporal analysis means comparing more than two observations of the same location over time, rather than a single before/after pair. Doing this can, in principle, reveal how change accumulates or fluctuates across several intervals rather than across just one span of time.

20.2 What this project actually implements

Independent intervals -- no tracking, no causal trend claim

This project selects several real observation dates for one location, spread across the real available time span of a Sentinel-2 archive search, and analyzes each **adjacent pair of dates completely independently** using the exact same two-image pipeline described elsewhere in this document. The underlying model has no mechanism to track a specific physical object or change across more than two images. A region flagged in one interval is **never** asserted to be the same underlying event as a region flagged in another interval -- they are separate, independent detections that happen to occupy nearby pixels. This project does not implement, and does not claim, temporal object tracking or a causal trend analysis.

20.3 Real, measured multi-temporal result

A real Sentinel-2 archive search for the same Pflugerville, Texas area, 2017-2024, with cloud cover under 5%, found 385 real candidate dates. Five dates were selected, spread across that real span, producing 4 independent adjacent-pair intervals:

Interval	Regions	Changed pixels	Changed area
2017-01-07 -> 2019-01-05	1	390	3.90 ha
2019-01-05 -> 2021-01-04	1	720	7.20 ha
2021-01-04 -> 2022-12-25	3	426	4.26 ha
2022-12-25 -> 2024-12-31	1	286	2.86 ha

Source: `outputs/multitemporal/temporal_report.json`, `docs/EVALUATION.md`. These four numbers must not be read as a rising/falling trend line for one physical change -- each is a separate detection over a different, non-overlapping pair of dates.

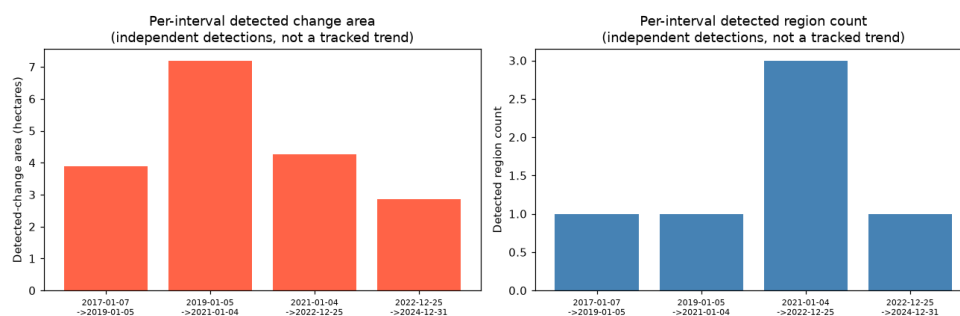


Figure 20.1 -- Real per-interval detected change area and region count, explicitly captioned as independent detections (source: `outputs/multitemporal/temporal_change_area.png`).

21. Real-World and Live Detection Workflow

21.1 The live upload-and-detect workflow

- The user uploads a before image and an after image (supported formats: PNG, JPG/JPEG, TIFF, BMP).
- Automatic input validation runs: dimension matching, an estimated registration offset (phase correlation), and a bright/low-saturation heuristic screen (Section 23).
- Preprocessing resizes both images to the model's input resolution (256x256) and normalizes them.
- The selected trained model runs inference, producing raw logits.
- A sigmoid converts the logits to a per-pixel probability map.
- The configured decision threshold converts the probability map into a binary prediction mask.
- Connected-component region extraction converts the mask into a list of individually described regions (Section 17), each scored for severity (Section 18).
- Results are visualized (before/after/mask/overlay/probability views) and summarized (region count, changed area, changed percentage), and can be exported.

21.2 The real-world Sentinel-2 demonstration

Separately from arbitrary user uploads, this project ran a documented, reproducible demonstration on real, independently sourced Sentinel-2 imagery (a Pflugerville, TX suburb, 2019-12-06 versus 2024-12-19, both near-zero cloud cover). Result: 1,621 of 65,536 pixels predicted changed (2.47%), 19 regions of at least 4 pixels. By visual inspection, the model's largest predicted region correctly corresponds to a real, visually confirmable new building complex; several smaller predicted regions could not be independently confirmed or ruled out as real change versus domain-gap artifacts. No accuracy metric was computed, because no ground truth exists for this scene (Section 12). Source: docs/REAL_WORLD_DEMO.md, outputs/real_world_demo/report.json.

21.3 Limitations that apply to any real-world or live-upload imagery

Factor	Why it matters here
Image resolution	The model was trained exclusively on 0.5 m/pixel imagery; Sentinel-2 (10 m/pixel) is 20x coarser -- a real, measured (Section 19) and honestly documented domain gap
Image registration	LEVIR-CD's pairs are pre-registered by the dataset authors; arbitrary uploads are not guaranteed to be aligned, and this project's robustness test (Section 13) measured a real IoU cost from a simulated misregistration
Satellite source / sensor	Different sensors have different radiometric processing; no correction for this difference has been attempted
Geographic coverage	All training data comes from central-Texas suburbs; performance on architecturally or geographically different regions is untested
Domain shift	Performance measured on the LEVIR-CD benchmark does not automatically transfer to a different sensor, resolution, or geography
Clouds / shadows	This project implements only a simple brightness/saturation heuristic screen (Section 23), not a validated cloud detector
Illumination / seasonal differences	Not explicitly corrected for; one anecdotal test case (a strong seasonal lighting difference) was correctly handled by the model, but this is one example, not a systematic robustness guarantee

22. Failure Cases

22.1 What is a failure case, and why it matters

A failure case is a specific, concrete example where a model's prediction was wrong -- a false positive (predicting change where none occurred) or a false negative (missing a real change). Documenting real failure cases, rather than only reporting aggregate metrics, is important because it shows exactly how and where a model can be wrong, which is essential for judging whether it is trustworthy for a specific real use case, and for guiding future improvement work.

22.2 Documented failure case: false positives on a genuinely no-change scene

Field	Description
Input	A held-out LEVIR-CD test scene with no real building change
Expected result	An almost-empty prediction mask (matching the empty ground-truth mask)
Model result	A small cluster of false-positive predictions, with no counterpart in the ground truth or in any other model's output on that same scene
False positives / negatives	A small number of false-positive pixels; no false negatives (there was no real change to miss)
Possible reason	Plausibly a lighting, seasonal, or fine-texture difference between the before and after image being misread as building change -- exactly the class of "apparent visual difference versus actual change" risk this project's own design principle warns about
Lesson learned	Even the best-performing model in this project is not immune to false positives on subtle apparent differences; this is disclosed rather than hidden, and is why a raw prediction should not be treated as certain without review

Source: docs/EXPERIMENTS.md ("Qualitative note"), observed on the Phase-8-era 30-epoch attention checkpoint's real prediction grid.

A second, reassuring real data point exists alongside this failure case: a different genuinely no-change test scene with a strong seasonal lighting/vegetation difference (before: dry/brown, after: green/lush) was correctly handled, producing only 3 tiny regions covering 0.05% of the tile. Both examples are reported as single, specific data points, not as a systematic robustness study.

23. Input Validation

src/realworld/validation.py implements real, computed checks on uploaded or externally sourced image pairs. These checks describe the input; they do not, and cannot, validate whether the resulting prediction is accurate.

Check	Method	Scientific status
Dimension match	Direct shape comparison of the two images	A hard, exact check -- not a heuristic
Registration-offset estimate	Phase correlation (cv2.phaseCorrelate) between the before and after image, flagged above a 3-pixel estimated shift	A real, standard signal-processing technique, used here as a diagnostic estimate only -- it does not align or correct the images, and can also be triggered by genuine large-scale change, not only misalignment
Resolution plausibility	Compares a known real pixel size (when available) against the training resolution, flagging imagery 5x coarser or more	A direct, exact numerical comparison -- only computed when a real pixel size is known, never guessed
Cloud / bright-region heuristic	Flags images where more than 5% of pixels are simultaneously bright and low-saturation	Explicitly a heuristic, not a validated cloud detector -- no labeled cloud-mask data exists in this project to validate it against; it will miss thin/translucent cloud and can false-positive on bright rooftops, sand, or snow

The required disclaimer

"Model trained on LEVIR-CD imagery. Performance on this imagery has not been independently validated."
This exact text is displayed wherever real-world or non-LEVIR-CD imagery is processed, in both the dashboard and the real-world demonstration script.

24. Why Streamlit

A necessary clarification

Streamlit itself runs locally on localhost by default -- it is a Python web-application framework, not a hosting service, and it can be run entirely on a local machine (as this project currently does) or deployed to a server. "Streamlit versus localhost" is not a meaningful technical distinction; the real question is why Streamlit was chosen as the application/dashboard framework, addressed below.

Streamlit was selected because it lets a data-science-oriented codebase (Python, PyTorch, NumPy, OpenCV, Pandas) become an interactive application with minimal separate frontend engineering. Concretely, in this project, Streamlit provides:

- Direct, in-process Python integration with the trained PyTorch model -- no separate API server or serialization layer was needed between the model and the interface.
- Built-in file-upload widgets, used directly for the before/after image upload workflow.
- Built-in interactive controls (sliders, selectboxes, number inputs) used for the model selector, decision threshold, and minimum region size.
- Built-in charting and dataframe display, used for the model-comparison tables/charts and the region tables.
- A multi-page navigation system (st.navigation), used to organize the dashboard into task-based pages without hand-building routing.
- Rapid iteration -- the entire interactive layer could be built and revised without writing a separate REST API, frontend JavaScript framework, or build pipeline, letting development focus on the model and evaluation, per this project's stated goal that the dashboard is a demonstration layer, never built ahead of a working, evaluated model.

This project currently runs the dashboard locally, launched with **streamlit run dashboard/app.py** and served on localhost by default; the same application could be deployed to a remote server without any change to its code.

25. The Dashboard

The dashboard is named "Satellite Change Intelligence" and is organized into six pages, focused on user tasks. This section describes only the user-facing functionality; internal development-phase labels are not part of the interface (Section 26 covers development history separately, briefly, and outside the main application).

Page	What the user can do	What they get
Overview	Nothing to configure -- a landing page	Real flagship benchmark metrics (IoU/Dice/F1/Accuracy), a plain-language capability list, and a quick-start guide
Change Detection	Upload a before/after pair; select a model, decision threshold, and minimum region size in the sidebar; run detection	Validation status, the predicted mask/overlay/probability map, a detection summary (region count, changed area/percentage, mean prediction probability), a sortable/filterable region table with severity scores, and CSV/JSON export
Model Analysis	Nothing to configure -- browses fixed, real results	The real 6-architecture comparison table and charts (parameters, inference time, IoU/Dice/Precision/Recall/F1/Accuracy) and the training-strategy comparison
Geospatial Intelligence	Downloads the GeoJSON export	The most recent real geospatial run: an interactive map of detected regions with real coordinates/area, and region-level detail
Temporal Analysis	Nothing to configure -- browses a fixed, real result	The most recent real multi-date analysis run, interval by interval, with the no-tracking disclaimer shown prominently
Diagnostics	Nothing to configure -- reference material	Full technical detail: input-validation methodology, selected-model internals, the probability/threshold explanation, a documented failure case, and the complete limitations list

Sidebar controls (visible on every page): model selection (8 trained models, including the research-only Transformer, clearly labeled), the decision threshold slider (defaulting to the validation-optimized value only for the model it was actually swept for), the minimum region size, and a compact display of the selected model's own real IoU/Dice.

26. Development History (Condensed)

The project was developed through a sequence of verified, individually committed stages. This section summarizes them into logical categories for context; it is not the focus of this document, and the dashboard itself does not expose phase numbers or implementation-status tables to end users -- that level of detail remains in `DEVELOPMENT_LOG.md` and `docs/` for anyone who wants it.

Category	What was added
Dataset & baseline	LEVIR-CD acquisition and verification, preprocessing pipeline, the baseline U-Net
Siamese architecture	The shared-encoder Siamese U-Net and its three feature-comparison modes
Attention & training improvements	Attention U-Net gates; early stopping, learning-rate scheduling, and longer training budgets; loss-function and hyperparameter experiments
Evaluation & robustness	Rigorous IoU/Dice/Precision/Recall/F1/Accuracy evaluation, prediction-probability visualization, threshold optimization, and controlled robustness testing
Region intelligence	Connected-component region extraction, geometry, and the analytical severity score
Geospatial intelligence	Real georeferenced Sentinel-2 access, polygon/area conversion, GeoJSON/GeoPackage export, and interactive mapping
Multi-temporal analysis	Independent-interval analysis across a real, multi-date Sentinel-2 sequence
Architecture research	A Transformer-based architecture, built and honestly compared against the CNN-based models
Real-world / input hardening	Registration-offset estimation, resolution-plausibility checks, and a cloud/overexposure heuristic for arbitrary uploaded imagery
Unified dashboard	Consolidation of every capability above into one cohesive, task-organized, professionally designed application

27. Real-World Applications

27.1 Currently supported capability

Given a pair of reasonably well-resolved, reasonably well-registered before/after satellite or aerial images, this system can detect and quantify likely **building change** -- new construction, demolition, or major structural modification -- and present the result with region-level statistics, an analytical severity ranking, and, where real georeferenced imagery is used, genuine geographic coordinates and area. This has direct, currently supported applicability to:

- Urban development monitoring -- tracking where new construction is occurring in a region over time.
- Construction-site progress or presence monitoring, at the resolution/coverage the model was trained for.
- A general-purpose demonstration/research platform for satellite-imagery change detection workflows (upload, inference, region analysis, geospatial export).

27.2 Potential future application (not currently supported)

These require capability this project does not currently have -- listed as potential, not as existing Disaster/damage assessment, infrastructure-specific monitoring, land-use-type change analysis, and environmental monitoring would all require either multi-class change classification or damage-severity ground-truth data that this project's binary building-change training labels do not provide (Section 28). They are listed here as directions the underlying approach could plausibly be extended toward, not as things the current model can already do.

28. Limitations

Every limitation below traces to a real, documented finding in the repository (docs/LIMITATIONS.md), not a generic disclaimer list.

28.1 Dataset limitations

- LEVIR-CD is binary building-change only, from 20 regions in and around Texas cities, 2002-2018. The model has never seen roads, vegetation, or water as a labeled change category, so it cannot classify change type.
- Strong class imbalance: only about 4.2-5.1% of pixels are labeled changed across the splits -- why accuracy alone is never reported as a standalone metric.
- Ground-truth masks required binarization (threshold 127) since raw mask files are not perfectly binary -- a reasonable but unvalidated choice.
- Single geographic region: all training data is from central-Texas suburbs; performance on architecturally or geographically different regions is untested.

28.2 Model / training limitations

- The five original (Phase 4/5/8) models were trained for a fixed 30-epoch budget; validation IoU was still trending upward at epoch 30 in most of those runs.
- No formal hyperparameter search was performed for those five models -- they share one recipe specifically so the architecture comparison would be apples-to-apples.
- Single run, single seed per experiment throughout this project -- no confidence intervals or variance estimates exist for any reported metric.
- GPU training on this project's hardware is not bit-exact reproducible even with a fixed seed, due to non-deterministic cuDNN convolution algorithms -- a documented, measured finding, not an assumption.
- Only the diff+concat Siamese comparison mode was ever combined with attention; the attention+diff and attention+concat combinations were not tried.
- The Transformer architecture underperforms every CNN variant tested (Section 6, Section 13) -- it is a research comparison only, not a production candidate.

28.3 Evaluation limitations

- All quantitative results are measured exclusively on the LEVIR-CD held-out test split -- a curated, pre-registered, single-sensor dataset. These numbers say nothing directly about performance on other imagery.
- False positives and false negatives are real and quantified, not hidden -- e.g. the baseline model's test-set confusion matrix records 125,241 false positives and 82,806 false negatives out of 8,388,608 total pixels.

28.4 Domain shift: benchmark versus real-world (the largest gap found)

- Sentinel-2 real-world imagery is 20x coarser than the training data (10 m/pixel versus LEVIR-CD's 0.5 m/pixel).
- Different sensor and radiometric processing between Sentinel-2 and the Google Earth composite imagery LEVIR-CD is built from -- no attempt has been made to quantify or correct for this.
- No ground truth exists for the real-world demonstration, so no accuracy metric can be or was computed for Sentinel-2 predictions.
- No independent re-registration or alignment-quality check was performed for the real-world Sentinel-2 imagery beyond its own standard georeferencing.

28.5 Multi-class change detection: not implemented, and why

A properly labeled multi-class change-detection dataset was required to implement this capability honestly -- LEVIR-CD's binary labels were never repurposed to fabricate classes. Three real candidate datasets were investigated (SECOND, HRSCD-Clean, xView2/xBD); none could be reliably obtained given the measured network conditions available at the time (the smallest viable candidate would have taken an estimated 2.4 days to download; the largest, an estimated 38 days). This is a genuine infrastructure constraint, documented rather than worked around by misusing the existing binary labels.

28.6 Real-world input validation: real checks, deliberately not more than that

- The registration-offset estimate is a diagnostic only -- it does not correct or align images, and can be triggered by genuine large-scale change, not only misalignment.
- The cloud/bright-region heuristic is explicitly not a validated cloud detector -- it will miss thin/translucent cloud and can false-positive on bright rooftops, sand, or snow.
- None of these checks validate prediction accuracy -- they describe the input only.

28.7 Not implemented (explicitly, never implied as working)

- Actual image co-registration/alignment correction (only an offset estimate exists).
- A validated, non-heuristic cloud/shadow detector.
- Change-type classification (building versus road versus vegetation versus water) -- no such labels exist in the training data.
- Physical-area estimation for non-LEVIR-CD imagery using the LEVIR-CD-specific pixel-size assumption -- deliberately not applied to Sentinel-2 predictions.
- Multi-seed statistical significance testing between architectures.
- Formal probability calibration (e.g. reliability diagrams, Expected Calibration Error) -- "prediction probability" is a raw sigmoid output only, never described as calibrated confidence.

29. Future Scope

Labeled clearly as future work -- none of this is an existing capability

Everything in this section is a direction for future development, not something the current system does.

- Multi-class / land-cover-aware change classification, if a suitable labeled dataset can be obtained (Devansh25/xview2, with real 4-class building-damage-severity labels, was identified as the strongest revisit candidate).
- A stronger, non-heuristic cloud/shadow detection model.
- Domain adaptation techniques to narrow the benchmark-versus-real-world performance gap.
- Larger and/or higher-resolution training datasets.
- True temporal sequence modeling (as opposed to this project's independent-interval analysis), enabling genuine change-trajectory or object-tracking claims.
- A hierarchical/multi-scale Transformer design (Swin-style), to test whether a more capable Transformer variant can close the gap to the CNN-based model.
- Uncertainty estimation and formal probability calibration.
- Deployment optimization and real-time inference.
- Deeper GIS integration beyond the current GeoJSON/GeoPackage export.

30. Complete System Workflow

The diagram below reflects the actual implemented pipeline, from raw input imagery through to the dashboard's presentation layer.

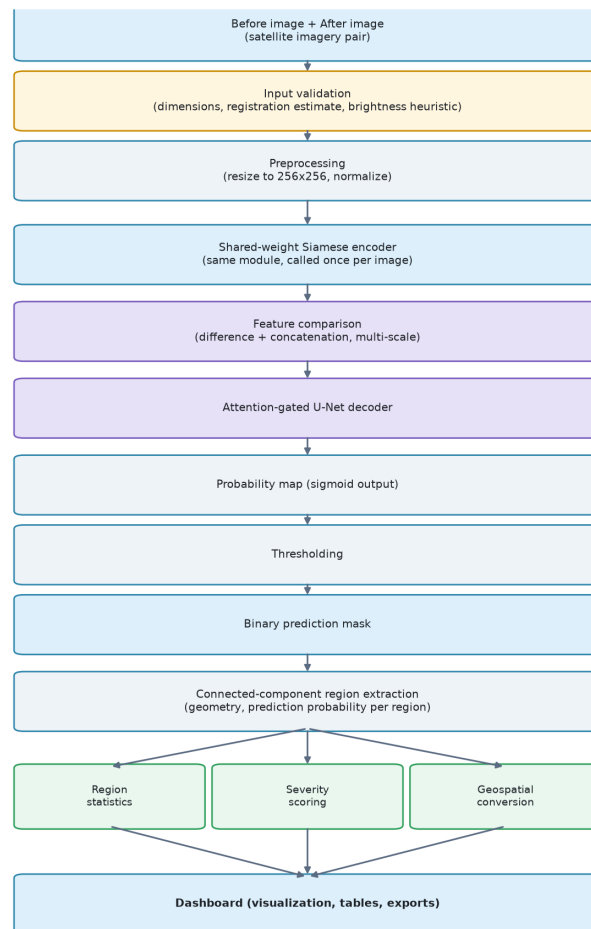


Figure 30.1 -- End-to-end pipeline as actually implemented in this project.

31. Project Interview and Viva -- Questions and Answers

Every answer below is grounded in this project's actual implementation and measured results, cross-referenced to the sections above.

31.1 Problem, dataset, and task

Q. What is the problem statement?

A. Given a before and an after satellite image of the same location, automatically detect and quantify where building change occurred, at the pixel level (Section 1).

Q. Why is satellite change detection important?

A. It replaces slow, inconsistent manual comparison of before/after imagery with an automated, consistent process, useful for monitoring urban development, construction, and land use at scale (Section 1, Section 27).

Q. What dataset did you use?

A. LEVIR-CD, a peer-reviewed benchmark building change-detection dataset (Chen and Shi, 2020) (Section 2).

Q. Where did the dataset come from?

A. The official distribution is Google Drive/Baidu Drive; this project acquired it through a documented Hugging Face mirror, since the official links have no reliable programmatic download path (Section 2).

Q. How large is the dataset?

A. 637 total image-pair samples: 445 for training, 64 for validation, 128 for testing (Section 2).

Q. What does LEVIR-CD contain?

A. 1024x1024-pixel before/after image pairs at 0.5 m/pixel resolution, with a binary building-change ground-truth mask for each pair, from 20 regions in Texas, imagery captured 2002-2018 (Section 2).

Q. What is a before image?

A. The earlier-dated satellite image in a pair -- the reference state against which change is measured (Section 3, Section 15).

Q. What is an after image?

A. The later-dated satellite image of the same location -- the state being checked for change against the before image (Section 3, Section 15).

Q. What is a ground-truth mask?

A. A human-annotated, pixel-level binary image marking exactly which pixels changed between the before and after image -- the label used to train and evaluate the model (Section 2).

Q. What is change detection?

A. The task of identifying differences between two observations of the same scene taken at different times (Section 5).

Q. Is this classification or segmentation?

A. Segmentation -- specifically binary semantic segmentation. The model predicts a label for every pixel, not a single label for the whole image (Section 3).

Q. Why is this a segmentation problem, not a classification problem?

A. Because the required output is a full spatial map of where change occurred, not a single yes/no answer for the whole image pair (Section 3).

31.2 Architecture

Q. Why did you use U-Net?

A. U-Net's encoder-decoder structure with skip connections is a well-established, effective design for pixel-level segmentation, recovering both high-level context and fine spatial detail (Section 5).

Q. Why Siamese U-Net specifically?

A. It lets the same, weight-shared encoder process the before and after image, guaranteeing both are analyzed with identical filters before their features are explicitly compared -- and this project measured it outperforming a non-Siamese baseline (Section 7).

Q. Why share encoder weights instead of using two separate encoders?

A. Sharing weights forces both images through identical filters, so a difference in the resulting features reflects a real image difference rather than a difference in how the two images were processed, and it uses the moderately sized training set (445 pairs) more efficiently than training two independent encoders (Section 7).

Q. What is the difference between diff, concat, and diff+concat comparison modes?

A. diff takes the absolute difference of the before/after features; concat stacks both feature maps side by side; diff+concat does both. diff+concat performed best in this project's measured comparison (Section 6, Section 13).

Q. What is attention, and why use it?

A. A mechanism that lets the network learn to weight feature-map locations by relevance rather than treating all locations equally; adding it improved every measured metric in this project's controlled comparison (Section 8).

Q. What is the difference between the plain and attention-gated U-Net here?

A. The attention-gated version re-weights every decoder skip connection using a learned gate before it is used, at a cost of about 724,000 extra parameters (Section 8).

Q. Did you try a Transformer architecture?

A. Yes -- a genuine self-attention Siamese encoder, trained and evaluated under the identical protocol as the CNN models. It underperformed every CNN variant (test IoU 0.3575), reported honestly rather than hidden (Section 6, Section 13).

Q. Why did the Transformer perform worse?

A. Consistent with the general finding that Vision Transformers usually need substantially more training data or large-scale pretraining than this project's 445 from-scratch training pairs provide (Section 6).

31.3 Output, probability, and thresholding

Q. What is a prediction mask?

A. The model's final binary output: a grid of 0s and 1s marking which pixels are predicted as changed (Section 15).

Q. What is a probability map?

A. The per-pixel probability (0 to 1) that a pixel changed, produced by applying sigmoid to the model's raw output, before any threshold is applied (Section 15).

Q. What is sigmoid, and why is it used here?

A. A function that squashes any real number into the range 0 to 1, letting the model's raw output be interpreted as a probability (Section 5, Section 15).

Q. What is thresholding?

A. Converting the continuous probability map into a binary mask by choosing a cutoff value -- pixels above the threshold are classified as changed (Section 15, Section 16).

Q. Is the prediction probability a calibrated confidence score?

A. No. It is explicitly described throughout this project as a raw sigmoid output, never as a calibrated confidence, since no calibration study (such as reliability diagrams or Expected Calibration Error) has been performed (Section 15, Section 28).

31.4 Evaluation metrics

Q. What is IoU?

A. Intersection over Union: the overlap between predicted and true changed regions divided by their union. It is this project's primary ranking metric (Section 11).

Q. What is Dice?

A. A closely related overlap metric, $2 \times TP / (2 \times TP + FP + FN)$ -- mathematically a monotonic transform of IoU, so it always ranks models identically to IoU (Section 11).

Q. What is F1 score?

A. The harmonic mean of precision and recall, balancing both types of error in one number (Section 11).

Q. What is precision?

A. Of the pixels predicted as changed, the fraction that actually changed (Section 11).

Q. What is recall?

A. Of the pixels that actually changed, the fraction the model correctly predicted (Section 11).

Q. Why is accuracy not enough on its own?

A. Because only about 4-5% of pixels are actually changed in this dataset, a model that always predicts no change would score over 94% accuracy while detecting nothing useful (Section 11).

31.5 Loss functions and optimization

Q. What is BCE loss?

A. Binary Cross-Entropy -- a loss that penalizes confident wrong per-pixel predictions, treating every pixel equally (Section 10).

Q. What is Dice loss?

A. A loss derived from the Dice overlap metric, which is not dominated by the majority (unchanged) class the way plain BCE can be (Section 10).

Q. Why combine BCE and Dice?

A. BCE provides a stable gradient from the start of training; Dice keeps training focused on the rare positive class. This combination measurably outperformed three tested alternatives (Focal+Dice, Weighted BCE+Dice, Tversky) in this project (Section 10, Section 13).

Q. What optimizer did you use?

A. Adam by default, with AdamW (Adam plus decoupled weight decay) tested as an alternative (Section 9).

Q. What is Adam?

A. An optimization algorithm that adapts the effective learning rate per parameter using running estimates of the gradient's mean and variance -- a widely used default for deep learning (Section 5).

Q. What is learning rate?

A. A number controlling how large each weight-update step is during training; this project's default is 0.0001 (Section 5, Section 9).

Q. What is a learning-rate scheduler?

A. A rule that automatically adjusts the learning rate during training; this project's best model uses ReduceLROnPlateau, halving the rate after 4 epochs without validation-IoU improvement (Section 5, Section 9).

Q. What is early stopping?

A. Automatically ending training once validation performance stops improving for a set number of epochs, while always retaining the best checkpoint seen (Section 5, Section 9).

Q. What is an epoch?

A. One complete pass through the entire training dataset (Section 5).

31.6 Results

Q. How many epochs were used for the best model?

A. A maximum of 100 configured epochs; training actually stopped at epoch 78 via early stopping, with the best checkpoint recorded at epoch 68 (Section 9, Section 14).

Q. How long did training take for the best model?

A. 3253.9 seconds (54.2 minutes), on a single NVIDIA RTX 4050 Laptop GPU (Section 14).

Q. Which model performed best?

A. The Siamese U-Net + Attention model trained with the 100-epoch budget, early stopping, and a learning-rate scheduler -- test IoU 0.7123 (Section 6, Section 13).

Q. What is the final accuracy?

A. 0.9830 on the held-out LEVIR-CD test set, for the recommended model -- reported alongside IoU/Dice/Precision/Recall/F1, never alone (Section 12).

Q. What is the final IoU?

A. 0.7123 (Section 12).

Q. What is the final Dice score?

A. 0.8320 (Section 12).

Q. Why did you compare so many architectures and configurations?

A. To make an evidence-based claim about which model is best, and to honestly report the configurations that did not help, rather than presenting only the final winner without context (Section 13).

31.7 Region, geospatial, and temporal analysis

Q. What is region-level analysis?

A. Grouping connected changed pixels into distinct regions and computing geometry (area, bounding box, perimeter, aspect ratio, density) and the model's own prediction probability for each one (Section 17).

Q. Why calculate region area and geometry?

A. A raw pixel mask does not tell a user how many distinct changes exist or how significant each one is; region-level statistics make the result interpretable (Section 17).

Q. What is geospatial analysis, in this project?

A. Converting detected pixel regions into real geographic polygons with true coordinates and area, using real georeferenced Sentinel-2 imagery -- never applied to plain LEVIR-CD PNGs, which have no coordinate system (Section 19).

Q. What is GeoJSON?

A. A standard, widely supported file format for representing geographic features (points, lines, polygons) with coordinates, used here to export detected change regions (Section 19).

Q. What is WGS84?

A. The standard geographic coordinate system (latitude/longitude) used by GPS and required by the GeoJSON specification (Section 19).

Q. What is UTM?

A. A projected, zone-based coordinate system in which distances and areas can be measured directly in meters -- used here to compute real region area before reprojecting to WGS84 for export (Section 19).

Q. What is multi-temporal analysis, in this project?

A. Analyzing a sequence of more than two real observation dates as independent adjacent-pair intervals -- not a tracked trend of one continuing change (Section 20).

Q. Does the system track the same change across multiple dates?

A. No. Each interval is an independent detection; the model has no mechanism to track a specific physical object or change across more than two images, and no such claim is made (Section 20).

31.8 Errors, limitations, and engineering

Q. What are false positives?

A. Pixels the model predicted as changed that did not actually change (Section 11, Section 22).

Q. What are false negatives?

A. Pixels that actually changed but the model failed to predict (Section 11, Section 22).

Q. What are failure cases, and why document them?

A. Specific, concrete examples of wrong predictions; documenting them shows exactly how and where a model can fail, which is essential for judging its trustworthiness (Section 22).

Q. What are the main limitations of this project?

A. Benchmark-only evaluation, a real domain gap to real-world imagery, no multi-class capability, single-seed training runs, and an underperforming (research-only) Transformer variant, among others -- documented in full in Section 28.

Q. Why Streamlit?

A. It gives direct Python integration with the trained model and built-in interactive widgets, avoiding a separate frontend framework, while still being runnable locally or deployed (Section 24).

Q. How does live inference work end to end?

A. Upload, validate, preprocess, run the model, apply sigmoid and thresholding, extract regions, score severity, then visualize and allow export -- with no ground truth and therefore no accuracy claim for the result (Section 21).

Q. What happens when a user uploads two images?

A. The images are checked for compatible dimensions and screened for registration/brightness issues, then run through the selected model to produce a mask, region table, and severity scores (Section 21, Section 23).

Q. What are real-world applications of this system?

A. Urban development monitoring and construction-site monitoring are currently supported; disaster assessment and land-use classification are future-scope applications requiring capability this project does not currently have (Section 27).

Q. What would you improve in the future?

A. Multi-class change classification (given a suitable dataset), a validated cloud detector, domain adaptation for real-world imagery, true temporal tracking, and formal probability calibration, among other items in Section 29.

32. Explain This Project in Two Minutes

A ready-to-use spoken explanation for a viva, interview, or presentation

"This project detects building change between two satellite images using deep learning. The problem: comparing before-and-after satellite imagery by hand does not scale, so I built a model that does it automatically. The dataset is LEVIR-CD, a peer-reviewed benchmark of 637 before/after image pairs with pixel-level building-change labels, split 445 training, 64 validation, 128 test. The model is a Siamese U-Net: a single shared-weight encoder processes the before and after image separately, their features are explicitly compared at multiple scales, and a U-Net decoder with attention-gated skip connections turns that comparison into a per-pixel change probability map, which is thresholded into a binary mask. I trained this with Adam, a combined BCE-plus-Dice loss, and, for the final model, early stopping and a learning-rate scheduler over up to 100 epochs, stopping at epoch 78 with the best checkpoint at epoch 68. On the held-out test set, it reaches an IoU of 0.7123 and a Dice score of 0.8320 -- and I confirmed this was actually the best configuration by also testing three alternative loss functions, four hyperparameter variants, and a Transformer-based architecture, none of which beat it. Beyond the core model, I built region-level analysis, an analytical severity score, real geospatial export using actual Sentinel-2 imagery with true coordinates, and a multi-date analysis mode, all served through a Streamlit dashboard. The main limitation is domain shift: the model was trained on 0.5 meter-per-pixel imagery, and real-world Sentinel-2 imagery is 20 times coarser, so I documented that gap honestly with a real, un-cherry-picked demonstration rather than claiming benchmark accuracy transfers to real-world imagery."

33. Explain This Project to a Non-Technical Person

In plain language

Imagine you have two photos taken from a satellite of the exact same neighborhood -- one from a few years ago, one from today. This project uses an artificial-intelligence program that has been shown thousands of example before-and-after photo pairs, each with the new buildings already marked, until it learned to spot new construction on its own. Now, when you give it two new photos, it draws a map showing exactly where it thinks something new was built, tells you how many separate changes it found and roughly how big each one is, and can even place those changes on a real map with real GPS coordinates if the photos come with location information. It only knows how to spot "a building appeared, disappeared, or changed" -- it was never taught to recognize new roads, trees, or bodies of water, so it will not try to identify those. And because it was trained on very sharp, zoomed-in photos, it is less reliable on blurrier, more zoomed-out satellite photos from a different source -- that limitation is explained openly rather than hidden.

34. Technical Architecture Summary

Category	Technology / method	Purpose
Dataset	LEVIR-CD (637 pairs, 0.5 m/pixel)	Labeled ground truth for training and benchmark evaluation
Deep learning framework	PyTorch 2.6.0 (CUDA 12.4)	Defines, trains, and runs every neural network in the project
Architecture	Siamese U-Net (shared encoder, U-Net decoder)	Explicit before/after feature comparison for pixel-level change segmentation
Attention	Additive attention gates (Attention U-Net style)	Re-weights decoder skip connections by relevance; measurably improved every metric
Loss	BCE + Dice	Stable early training plus robustness to class imbalance
Optimizer	Adam (AdamW tested)	Updates model weights from computed gradients
Scheduler	ReduceLROnPlateau	Reduces the learning rate once validation IoU plateaus
Evaluation	IoU, Dice, Precision, Recall, F1, Accuracy (confusion-matrix accumulated)	Rigorous, imbalance-aware performance measurement
Computer vision	OpenCV, Pillow, NumPy	Image I/O, resizing, contour/geometry extraction
Geospatial	Rasterio, Shapely, PyProj, GeoPandas, Folium, pystac-client	Real georeferenced imagery access, polygon/area conversion, and interactive mapping
Visualization	Matplotlib, Pandas	Training curves, comparison charts, and tabular displays
Dashboard	Streamlit 1.62.0	Interactive upload, inference, and analysis interface
Deployment / execution	Local process (streamlit run), CUDA GPU inference	Runs the trained model and dashboard on the project's own hardware
Storage	YAML configs, JSON/CSV metrics, PyTorch checkpoint files	Reproducible configuration and measured-result storage
Version control	Git / GitHub	Full commit history of every development stage

35. Current Project Status

Capability	Status
Binary building-change detection (Siamese U-Net + Attention)	Implemented, measured
Baseline U-Net and Siamese comparison-mode ablation (diff/concat/diff+concat)	Implemented, measured
Advanced training strategy (early stopping, LR scheduling, longer budgets)	Implemented, measured
Loss-function and hyperparameter experiments	Implemented, measured
Prediction probability visualization	Implemented, measured
Threshold optimization	Implemented, measured -- found the model threshold-insensitive
Robustness testing (noise/brightness/contrast/shift)	Implemented, measured -- found real sensitivity to darkening/contrast/misregistration
Region-level geometry and statistics	Implemented, measured
Change severity scoring	Implemented -- explicitly analytical, not ground truth
Geospatial analysis (real coordinates, GeoJSON/GeoPackage export, interactive map)	Implemented, measured
Multi-temporal (more than two date) analysis	Implemented, measured -- independent intervals only, no tracking
Transformer-based architecture	Experimental / research comparison only -- underperforms the CNN model
Real-world (Sentinel-2) demonstration	Implemented -- qualitative demonstration only, no accuracy metric possible
Input validation (dimensions, registration estimate, brightness heuristic)	Implemented -- diagnostics and heuristics, not corrections or validated detectors
Unified dashboard	Implemented
Multi-class / change-type classification	Not implemented -- no suitable labeled dataset could be reliably obtained
Validated (non-heuristic) cloud/shadow detection	Not implemented
Formal probability calibration	Not implemented
Multi-seed variance / confidence-interval reporting	Not implemented
Domain adaptation, true temporal tracking, larger/higher-resolution datasets	Future work -- not implemented

36. References

- Chen, H. and Shi, Z. (2020). "A Spatial-Temporal Attention-Based Method and a New Dataset for Remote Sensing Image Change Detection." *Remote Sensing*, 12(10):1662. (LEVIR-CD dataset; official project page: justchenhao.github.io/LEVIR)
- Ronneberger, O., Fischer, P., and Brox, T. (2015). "U-Net: Convolutional Networks for Biomedical Image Segmentation." (U-Net architecture)
- Oktay, O. et al. (2018). "Attention U-Net: Learning Where to Look for the Pancreas." (Attention-gate mechanism used in this project's Attention U-Net)
- Lin, T-Y. et al. (2017). "Focal Loss for Dense Object Detection." (Focal loss, tested as an alternative loss function in this project)
- Salehi, S. S. M., Erdogmus, D., and Gholipour, A. (2017). "Tversky Loss Function for Image Segmentation Using 3D Fully Convolutional Deep Networks." (Tversky loss, tested as an alternative loss function in this project)
- PyTorch. Paszke, A. et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library." (Deep learning framework used throughout this project)
- Streamlit documentation, streamlit.io (dashboard framework)
- Earth Search STAC API, Element84 (earth-search.aws.element84.com) -- real-world Sentinel-2 L2A imagery access used in this project's geospatial and multi-temporal analysis